一键烧写 QT 程序到开发板

之前我们写好一个程序，都是先交叉编译完，然后在通过 nfs/tftp 或者其他的方法拷贝到开发板，然后在手动执行，如果我们调试程序很频繁，那我们每次都重新拷贝到开发板，那这样就太麻烦了，有没有方便一点的办法呢，这个就是接下来我们要介绍的一种方法，一键烧写 QT 程序到开发板，我们只需要在 qtcreater 上点击一下编译，那么 QT 程序会自动到开发板上来运行，是不是非常的方便?

一．在开发板上移植 ssh

要想实现一键烧写，那么我们必须就先要在开发板上移植一个 ssh，本次实验我们使用的是 busybox 文件系统，这个文件系统并不支持 ssh，所以我们要先移植一个上去。接下来，我们一起来移植一下。

移植环境:

编译器保持和编译文件系统和 QT 的编译器一致:

arm-2014.05-29-arm-none-linux-gnueabi-i686-pc-linux-gnu.tar.bz2

Ubuntu 版本：Ubuntu16

（1）移植 zlib

从迅为提供的资料中拷贝“ zlib-1.2.3.tar.gz” 到 Ubuntu 系统， 使用命令“ tar -vxf zlib-1.2.3.tar.gz” 解压， 得到“zlib-1.2.3” 文件夹， 使用命令“cd zlib-1.2.3” 进入“ zlib-1.2.3”文件夹， 如下图

```
root@ubuntu:/home/ssh# ls
zlib-1.2.3  zlib-1.2.3.tar.gz
root@ubuntu:/home/ssh# cd zlib-1.2.3
root@ubuntu:/home/ssh/zlib-1.2.3# ls
adler32.c      crc32.c     INDEX       inftrees.h    qnx          zlib.3
algorithm.txt  crc32.h     infback.c   Makefile      README       zlib.h
amiga          deflate.c   inffast.c   Makefile.in   trees.c      zutil.c
as400          deflate.h   inffast.h   make_vms.com  trees.h      zutil.h
ChangeLog      example.c   inffixed.h  minigzip.c    uncompr.c
compress.c     examples    inflate.c   msdos         win32
configure      FAQ         inflate.h   old           zconf.h
contrib        gzio.c      inftrees.c  projects      zconf.in.h
```

接着使用命令“./configure --prefix=/home/ssh/ssh_install/zlib-1.2.3” ， 设置配置文件，

其中--prefix=/home/ssh/ssh_install/zlib-1.2.3 指定了 zlib 的安装路径。

如下图所示 ：

```
root@ubuntu:/home/ssh/zlib-1.2.3# ./configure --prefix=/home/ssh/ssh_install/zlib-1.2.3
Checking for gcc...
Building static library libz.a version 1.2.3 with gcc.
Checking for unistd.h... Yes.
Checking whether to use vs[n]printf() or s[n]printf()... using vs[n]printf()
Checking for vsnprintf() in stdio.h... Yes.
Checking for return value of vsnprintf()... Yes.
Checking for errno.h... Yes.
Checking for mmap support... Yes.
```

指定了安装路径

使用命令 “ vim Makefile” 修改编译参数， 将所有 gcc 修改为 “ arm-none-linux-gnueabi-gcc”

如下图所示

```
CC=arm-none-linux-gnueabi-gcc

CFLAGS=-O3 -DUSE_MMAP
#CFLAGS=-O -DMAX_WBITS=14 -DMAX_MEM_LEVEL=7
#CFLAGS=-g -DDEBUG
#CFLAGS=-O3 -Wall -Wwrite-strings -Wpointer-arith -Wconversion \
#           -Wstrict-prototypes -Wmissing-prototypes

LDFLAGS=-L. libz.a
LDSHARED=arm-none-linux-gnueabi-gcc
CPP=arm-none-linux-gnueabi-gcc -E

LIBS=libz.a
SHAREDLIB=libz.so
SHAREDLIBV=libz.so.1.2.3
SHAREDLIBM=libz.so.1
```

接着使用编译命令“make” ， 如下图所示

```
root@ubuntu:/home/ssh/zlib-1.2.3# make
arm-none-linux-gnueabi-gcc -O3 -DUSE_MMAP   -c -o example.o example.c
arm-none-linux-gnueabi-gcc -O3 -DUSE_MMAP   -c -o adler32.o adler32.c
arm-none-linux-gnueabi-gcc -O3 -DUSE_MMAP   -c -o compress.o compress.c
arm-none-linux-gnueabi-gcc -O3 -DUSE_MMAP   -c -o crc32.o crc32.c
arm-none-linux-gnueabi-gcc -O3 -DUSE_MMAP   -c -o gzio.o gzio.c
arm-none-linux-gnueabi-gcc -O3 -DUSE_MMAP   -c -o uncompr.o uncompr.c
arm-none-linux-gnueabi-gcc -O3 -DUSE_MMAP   -c -o deflate.o deflate.c
^[^Aarm-none-linux-gnueabi-gcc -O3 -DUSE_MMAP   -c -o trees.o trees.c
arm-none-linux-gnueabi-gcc -O3 -DUSE_MMAP   -c -o zutil.o zutil.c
arm-none-linux-gnueabi-gcc -O3 -DUSE_MMAP   -c -o inflate.o inflate.c
arm-none-linux-gnueabi-gcc -O3 -DUSE_MMAP   -c -o infback.o infback.c
arm-none-linux-gnueabi-gcc -O3 -DUSE_MMAP   -c -o inftrees.o inftrees.c
arm-none-linux-gnueabi-gcc -O3 -DUSE_MMAP   -c -o inffast.o inffast.c
ar rc libz.a adler32.o compress.o crc32.o gzio.o uncompr.o deflate.o trees.o zutil.o inflate.o infback.o inftrees.o inffast.o
arm-none-linux-gnueabi-gcc -O3 -DUSE_MMAP -o example example.o -L. libz.a
arm-none-linux-gnueabi-gcc -O3 -DUSE_MMAP   -c -o minigzip.o minigzip.c
arm-none-linux-gnueabi-gcc -O3 -DUSE_MMAP -o minigzip minigzip.o -L. libz.a
```

执行 make 无报错后， 接着使用安装命令“make install” ， 如下图所示

```
root@ubuntu:/home/ssh/zlib-1.2.3# make install
cp zlib.h zconf.h /home/ssh/ssh_install/zlib-1.2.3/include
chmod 644 /home/ssh/ssh_install/zlib-1.2.3/include/zlib.h /home/ssh/ssh_install/zlib-1.2.3/include/zconf.h
cp libz.a /home/ssh/ssh_install/zlib-1.2.3/lib
cd /home/ssh/ssh_install/zlib-1.2.3/lib; chmod 755 libz.a
cd /home/ssh/ssh_install/zlib-1.2.3/lib; if test -f libz.so.1.2.3; then \
  rm -f libz.so libz.so.1; \
  ln -s libz.so.1.2.3 libz.so; \
  ln -s libz.so.1.2.3 libz.so.1; \
  (ldconfig || true)  >/dev/null 2>&1; \
fi
cp zlib.3 /home/ssh/ssh_install/zlib-1.2.3/share/man/man3
chmod 644 /home/ssh/ssh_install/zlib-1.2.3/share/man/man3/zlib.3
root@ubuntu:/home/ssh/zlib-1.2.3#
```

现在已经安装好了 zlib 库， 在后面编译 SSH 的时候， 需要用到 zlib 库， 作者这里的路径是“/home/ssh/ssh_install/zlib-1.2.3” ， 后面配置的时候需要对应生成的库文件，如下图所示

```
root@ubuntu:/home/ssh/zlib-1.2.3# ls ../ssh_install/zlib-1.2.3/
include  lib  share
root@ubuntu:/home/ssh/zlib-1.2.3#
root@ubuntu:/home/ssh/zlib-1.2.3#
```

（2）移植 **openssl**

接着来移植 openssl 库， 拷贝“openssl-0.9.8h.tar.gz” 到 Ubuntu 系统， 使用命令“tar -vxf openssl-0.9.8h.tar.gz” 解压， 得到“openssl-0.9.8h” 文件夹， 使用命令“cd openssl-0.9.8h”进入“openssl-0.9.8h” 文件夹， 如下图所示。

```
root@ubuntu:/home/ssh# ls
openssl-0.9.8h  openssl-0.9.8h.tar.gz  ssh_install  zlib-1.2.3  zlib-1.2.3.tar.gz
root@ubuntu:/home/ssh# cd openssl-0.9.8h
root@ubuntu:/home/ssh/openssl-0.9.8h# ls
apps                                   Configure  include        INSTALL.W32   Makefile.shared  perl           times
bugs                                   crypto     INSTALL        INSTALL.W64   makevms.com      PROBLEMS       tools
certs                                  demos      install.com    INSTALL.WCE   ms               README         util
ChangeLog.0_9_7-stable_not-in-head     doc        INSTALL.DJGPP  LICENSE       Netware          README.ASN1    VMS
ChangeLog.0_9_7-stable_not-in-head_FIPS engines   INSTALL.MacOS  MacOS         NEWS             README.ENGINE
CHANGES                                e_os2.h    INSTALL.NW     Makefile      openssl.doxy     shlib
CHANGES.SSLeay                         e_os.h     INSTALL.OS2    Makefile.bak  openssl.spec     ssl
config                                 FAQ        INSTALL.VMS    Makefile.org  os2              test
root@ubuntu:/home/ssh/openssl-0.9.8h#
```

接着使用命令“./Configure linux-elf-arm -DB_ENDIAN linux:'arm-none-linux -gnueabi-gcc' --prefix=/home/ssh/ssh_install/openssl-0.9.8h” 配置编译文件， 如下图所示。

```
root@ubuntu:/home/ssh/openssl-0.9.8h# ./Configure linux-elf-arm -DB_ENDIAN linux:'arm-none-linux-gnueabi-gcc' --prefix=/home/ssh/
ssh_install/openssl-0.9.8h
```

使用编译命令“make” 编译完成， 如下图所示

```
-L//'; fi; done | uniq`; LIBPATH=`echo $LIBPATH | sed -e 's/ /:/g'`; LD_LIBRARY_PATH=$LIBPATH:$LD_LIBRARY_PATH ${LDCMD} ${LDFLAGS
} -o ${APPNAME:=rsa_test} rsa_test.o ${LIBDEPS} )
make[2]: Leaving directory '/home/ssh/openssl-0.9.8h/test'
arm-none-linux-gnueabi-gcc -I.. -I../include  -DOPENSSL_THREADS  -DB_ENDIAN    -c -o evp_test.o evp_test.c
make[2]: Entering directory '/home/ssh/openssl-0.9.8h/test'
( :; LIBDEPS="${LIBDEPS:--L.. -lssl -L.. -lcrypto  }"; LDCMD="${LDCMD:-arm-none-linux-gnueabi-gcc}"; LDFLAGS="${LDFLAGS:--DOPENSS
L_THREADS  -DB_ENDIAN }"; LIBPATH=`for x in $LIBDEPS; do if echo $x | grep '^ *-L' > /dev/null 2>&1; then echo $x | sed -e 's/^ *
-L//'; fi; done | uniq`; LIBPATH=`echo $LIBPATH | sed -e 's/ /:/g'`; LD_LIBRARY_PATH=$LIBPATH:$LD_LIBRARY_PATH ${LDCMD} ${LDFLAGS
} -o ${APPNAME:=evp_test} evp_test.o ${LIBDEPS} )
make[2]: Leaving directory '/home/ssh/openssl-0.9.8h/test'
arm-none-linux-gnueabi-gcc -I.. -I../include  -DOPENSSL_THREADS  -DB_ENDIAN    -c -o igetest.o igetest.c
make[2]: Entering directory '/home/ssh/openssl-0.9.8h/test'
( :; LIBDEPS="${LIBDEPS:--L.. -lssl -L.. -lcrypto  }"; LDCMD="${LDCMD:-arm-none-linux-gnueabi-gcc}"; LDFLAGS="${LDFLAGS:--DOPENSS
L_THREADS  -DB_ENDIAN }"; LIBPATH=`for x in $LIBDEPS; do if echo $x | grep '^ *-L' > /dev/null 2>&1; then echo $x | sed -e 's/^ *
-L//'; fi; done | uniq`; LIBPATH=`echo $LIBPATH | sed -e 's/ /:/g'`; LD_LIBRARY_PATH=$LIBPATH:$LD_LIBRARY_PATH ${LDCMD} ${LDFLAGS
} -o ${APPNAME:=igetest} igetest.o ${LIBDEPS} )
make[2]: Leaving directory '/home/ssh/openssl-0.9.8h/test'
arm-none-linux-gnueabi-gcc -I.. -I../include  -DOPENSSL_THREADS  -DB_ENDIAN    -c -o dummytest.o dummytest.c
make[2]: Entering directory '/home/ssh/openssl-0.9.8h/test'
( :; LIBDEPS="${LIBDEPS:--L.. -lssl -L.. -lcrypto  }"; LDCMD="${LDCMD:-arm-none-linux-gnueabi-gcc}"; LDFLAGS="${LDFLAGS:--DOPENSS
L_THREADS  -DB_ENDIAN }"; LIBPATH=`for x in $LIBDEPS; do if echo $x | grep '^ *-L' > /dev/null 2>&1; then echo $x | sed -e 's/^ *
-L//'; fi; done | uniq`; LIBPATH=`echo $LIBPATH | sed -e 's/ /:/g'`; LD_LIBRARY_PATH=$LIBPATH:$LD_LIBRARY_PATH ${LDCMD} ${LDFLAGS
} -o ${APPNAME:=dummytest} dummytest.o ${LIBDEPS} )
make[2]: Leaving directory '/home/ssh/openssl-0.9.8h/test'
make[1]: Leaving directory '/home/ssh/openssl-0.9.8h/test'
making all in tools...
make[1]: Entering directory '/home/ssh/openssl-0.9.8h/tools'
make[1]: Nothing to be done for 'all'.
make[1]: Leaving directory '/home/ssh/openssl-0.9.8h/tools'
root@ubuntu:/home/ssh/openssl-0.9.8h#
```

最后使用安装命令“make install” ， 如果在安装过程提示以下错误，如下图所示：

```
installing man1/ecparam.1
installing man1/enc.1
installing man1/errstr.1
installing man1/gendsa.1
installing man1/genrsa.1
installing man1/nseq.1
installing man1/ocsp.1
installing man1/openssl.1
installing man1/passwd.1
installing man1/pkcs12.1
installing man1/pkcs7.1
installing man1/pkcs8.1
installing man1/rand.1
installing man1/req.1
installing man1/rsa.1
installing man1/rsautl.1
installing man1/s_client.1
installing man1/s_server.1
installing man1/s_time.1
installing man1/sess_id.1
installing man1/smime.1
smime.pod around line 272: Expected text after =item, not a number
smime.pod around line 276: Expected text after =item, not a number
smime.pod around line 280: Expected text after =item, not a number
smime.pod around line 285: Expected text after =item, not a number
smime.pod around line 289: Expected text after =item, not a number
POD document had syntax errors at /usr/bin/pod2man line 68.
Makefile:544: recipe for target 'install_docs' failed
make: *** [install_docs] Error 255
root@ubuntu:/home/ssh/openssl-0.9.8h#
```

执行命令 rm -f /usr/bin/pod2man 可以解决，如下图所示：

```
smime.pod around line 272: Expected text after =item, not a number
smime.pod around line 276: Expected text after =item, not a number
smime.pod around line 280: Expected text after =item, not a number
smime.pod around line 285: Expected text after =item, not a number
smime.pod around line 289: Expected text after =item, not a number
POD document had syntax errors at /usr/bin/pod2man line 68.
Makefile:544: recipe for target 'install_docs' failed
make: *** [install_docs] Error 255
root@ubuntu:/home/ssh/openssl-0.9.8h# rm -f /usr/bin/pod2man
root@ubuntu:/home/ssh/openssl-0.9.8h#
```

然后再次执行 make install 即可安装成功，如下图所示：

```
make[1]: Entering directory '/home/ssh/openssl-0.9.8h/ssl'
make[1]: Leaving directory '/home/ssh/openssl-0.9.8h/ssl'
making install in engines...
make[1]: Entering directory '/home/ssh/openssl-0.9.8h/engines'
make[1]: Leaving directory '/home/ssh/openssl-0.9.8h/engines'
making install in apps...
make[1]: Entering directory '/home/ssh/openssl-0.9.8h/apps'
installing openssl
installing CA.sh
installing CA.pl
make[1]: Leaving directory '/home/ssh/openssl-0.9.8h/apps'
making install in test...
make[1]: Entering directory '/home/ssh/openssl-0.9.8h/test'
make[1]: Nothing to be done for 'install'.
make[1]: Leaving directory '/home/ssh/openssl-0.9.8h/test'
making install in tools...
make[1]: Entering directory '/home/ssh/openssl-0.9.8h/tools'
make[1]: Leaving directory '/home/ssh/openssl-0.9.8h/tools'
installing libcrypto.a
installing libssl.a
cp libcrypto.pc /home/ssh/ssh_install/openssl-0.9.8h/lib/pkgconfig
chmod 644 /home/ssh/ssh_install/openssl-0.9.8h/lib/pkgconfig/libcrypto.pc
cp libssl.pc /home/ssh/ssh_install/openssl-0.9.8h/lib/pkgconfig
chmod 644 /home/ssh/ssh_install/openssl-0.9.8h/lib/pkgconfig/libssl.pc
cp openssl.pc /home/ssh/ssh_install/openssl-0.9.8h/lib/pkgconfig
chmod 644 /home/ssh/ssh_install/openssl-0.9.8h/lib/pkgconfig/openssl.pc
root@ubuntu:/home/ssh/openssl-0.9.8h#
```

（3）移植 **ssh**

接着来移植 openssh， 拷贝“ openssh-4.6p1.tar.gz” 到 Ubuntu 系统， 使用命令“ tar -vxfopenssh-4.6p1.tar.gz” 解压， 得到“openssh-4.6p1” 文件夹， 使用命令“cd openssh-4.6p1/” 进入“openssh-4.6p1” 文件夹， 如下图所示

```
root@ubuntu:/home/ssh# cd openssh-4.6p1
root@ubuntu:/home/ssh/openssh-4.6p1# ls
aclocal.m4          auth-sia.h      deattack.h      loginrec.c      packet.h        sftp.c          ssh.h
acss.c              auth-skey.c     defines.h       loginrec.h      pathnames.h     sftp-client.c   ssh-keygen.0
acss.h              bufaux.c        dh.c            logintest.c     platform.c      sftp-client.h   ssh-keygen.1
atomicio.c          bufbn.c         dh.h            mac.c           platform.h      sftp-common.c   ssh-keygen.c
atomicio.h          buffer.c        dispatch.c      mac.h           progressmeter.c sftp-common.h   ssh-keyscan.0
audit-bsm.c         buffer.h        dispatch.h      Makefile.in     progressmeter.h sftp-glob.c     ssh-keyscan.1
audit.c             buildpkg.sh.in  dns.c            match.c         readconf.c      sftp.h          ssh-keyscan.c
audit.h             canohost.c      dns.h           match.h         readconf.h      sftp-server.0   ssh-keysign.0
auth1.c             canohost.h      entropy.c       md5crypt.c      README          sftp-server.8   ssh-keysign.8
auth2.c             ChangeLog       entropy.h       md5crypt.h      README.dns      sftp-server.c   ssh-keysign.c
```

这里看一下 ssh 的 configure 的帮助文件， 如下图所示， 使用“ ./configure --help

ssh 的 configure 文件中需要配置的参数
--host 表示编译出来要运行的平台
--with-libs 表示需要的额外的库文件;
--with-zlib 表示库文件 zlib 的 instal 路径 l;
--with-ssl-dir 表示 OpenSSl 文件的 install 路径;
--disable-etc-default-login 表示不使用当前环境变量的编译器， 所以后面配置参数时， 最后直接配置 CC 和 AR 这两个编译器;
CC 表示设置使用的编译器;
AR 表示设置使用的编译器的路径（蛮奇怪， 配置文件中似乎没有这个参数， 用户可以执行验证下）

接着使用配置命令， 命令比较长， 作者使用颜色给大家分段:

```
./configure --host=arm-none-linux --with-libs
--with-zlib=/home/ssh/ssh_install/zlib-1.2.3
--with-ssl-dir=/home/ssh/ssh_install/openssl-0.9.8h
--disable-etc-default-login CC=/usr/local/arm/arm-2014.05/bin/arm-none-linux-gnueabi-gcc
AR=/usr/local/arm/arm-2014.05/bin/arm-none-linux-gnueabi-ar
```

如下图所示:

```
root@ubuntu:/home/ssh/openssh-4.6p1# ./configure --host=arm-none-linux --with-libs --with-zlib=/home/ssh/ssh_install/zlib-1.2.3 --
-with-ssl-dir=//home/ssh/ssh_install/openssl-0.9.8h --with-ssl-dir=/home/ssh/ssh_install/openssl-0.9.8h AR=/usr/local/arm/arm-201
4.05/bin/arm-none-linux-gnueabi-ar
```

执行成功如下图所示:

```
                    User binaries: /usr/local/bin
                  System binaries: /usr/local/sbin
              Configuration files: /usr/local/etc
                  Askpass program: /usr/local/libexec/ssh-askpass
                    Manual pages: /usr/local/share/man/manX
                        PID file: /var/run
  Privilege separation chroot path: /var/empty
            sshd default user PATH: /usr/bin:/bin:/usr/sbin:/sbin:/usr/local/bin
                    Manpage format: doc
                       PAM support: no
                  OSF SIA support: no
                KerberosV support: no
                  SELinux support: no
                Smartcard support: no
                    S/KEY support: no
              TCP Wrappers support: no
              MD5 password support: no
                   libedit support: no
  Solaris process contract support: no
      IP address in $DISPLAY hack: no
          Translate v4 in v6 hack: yes
                 BSD Auth support: no
              Random number source: OpenSSL internal ONLY

              Host: arm-none-linux-gnu
          Compiler: /usr/local/arm/arm-2014.05/bin/arm-none-linux-gnueabi-gcc
    Compiler flags: -g -O2 -Wall -Wpointer-arith -Wuninitialized -Wsign-compare -Wno-pointer-sign
Preprocessor flags: -I/home/ssh/ssh_install/openssl-0.9.8h/include -I/home/ssh/ssh_install/zlib-1.2.3/include
      Linker flags: -L/home/ssh/ssh_install/openssl-0.9.8h/lib -L/home/ssh/ssh_install/zlib-1.2.3/lib
         Libraries:   -lcrypto -lutil -lz -lnsl  -lcrypt -lresolv -lresolv

root@ubuntu:/home/ssh/openssh-4.6p1#
```

使用编译命令“make” ， 编译完成， 如下图所示

```
H_PROGRAM=\"/usr/local/bin/ssh\" -D_PATH_SSH_ASKPASS_DEFAULT=\"/usr/local/libexec/ssh-askpass\" -D_PATH_SFTP_SERVER=\"/usr/local/libexec/
sftp-server\" -D_PATH_SSH_KEY_SIGN=\"/usr/local/libexec/ssh-keysign\" -D_PATH_SSH_PIDDIR=\"/var/run\" -D_PATH_PRIVSEP_CHROOT_DIR=\"/var/e
mpty\" -DSSH_RAND_HELPER=\"/usr/local/libexec/ssh-rand-helper\" -DHAVE_CONFIG_H -c sftp-server.c
sftp-server.c: In function 'main':
sftp-server.c:1244:18: warning: comparison between 'LogLevel' and 'enum <anonymous>' [-Wenum-compare]
    if (log_level == SYSLOG_FACILITY_NOT_SET)
                  ^
/usr/local/arm/arm-2014.05/bin/arm-none-linux-gnueabi-gcc -o sftp-server sftp-server.o sftp-common.o -L. -Lopenbsd-compat/ -L/home/ssh/ss
h_install/openssl-0.9.8h/lib -L/home/ssh/ssh_install/zlib-1.2.3/lib  -lssh -lopenbsd-compat -lcrypto -lutil -lz -lnsl  -lcrypt -lresolv -
lresolv
/usr/local/arm/arm-2014.05/bin/arm-none-linux-gnueabi-gcc -g -O2 -Wall -Wpointer-arith -Wuninitialized -Wsign-compare -Wno-pointer-sign
-I. -I. -I/home/ssh/ssh_install/openssl-0.9.8h/include -I/home/ssh/ssh_install/zlib-1.2.3/include  -DSSHDIR=\"/usr/local/etc\" -D_PATH_SS
H_PROGRAM=\"/usr/local/bin/ssh\" -D_PATH_SSH_ASKPASS_DEFAULT=\"/usr/local/libexec/ssh-askpass\" -D_PATH_SFTP_SERVER=\"/usr/local/libexec/
sftp-server\" -D_PATH_SSH_KEY_SIGN=\"/usr/local/libexec/ssh-keysign\" -D_PATH_SSH_PIDDIR=\"/var/run\" -D_PATH_PRIVSEP_CHROOT_DIR=\"/var/e
mpty\" -DSSH_RAND_HELPER=\"/usr/local/libexec/ssh-rand-helper\" -DHAVE_CONFIG_H -c sftp-client.c
sftp-client.c: In function 'do_realpath':
sftp-client.c:589:10: warning: variable 'a' set but not used [-Wunused-but-set-variable]
  Attrib *a;
          ^
sftp-client.c: In function 'do_readlink':
sftp-client.c:695:10: warning: variable 'a' set but not used [-Wunused-but-set-variable]
  Attrib *a;
          ^
/usr/local/arm/arm-2014.05/bin/arm-none-linux-gnueabi-gcc -g -O2 -Wall -Wpointer-arith -Wuninitialized -Wsign-compare -Wno-pointer-sign
-I. -I. -I/home/ssh/ssh_install/openssl-0.9.8h/include -I/home/ssh/ssh_install/zlib-1.2.3/include  -DSSHDIR=\"/usr/local/etc\" -D_PATH_SS
H_PROGRAM=\"/usr/local/bin/ssh\" -D_PATH_SSH_ASKPASS_DEFAULT=\"/usr/local/libexec/ssh-askpass\" -D_PATH_SFTP_SERVER=\"/usr/local/libexec/
sftp-server\" -D_PATH_SSH_KEY_SIGN=\"/usr/local/libexec/ssh-keysign\" -D_PATH_SSH_PIDDIR=\"/var/run\" -D_PATH_PRIVSEP_CHROOT_DIR=\"/var/e
mpty\" -DSSH_RAND_HELPER=\"/usr/local/libexec/ssh-rand-helper\" -DHAVE_CONFIG_H -c sftp-glob.c
/usr/local/arm/arm-2014.05/bin/arm-none-linux-gnueabi-gcc -o sftp progressmeter.o sftp.o sftp-client.o sftp-common.o sftp-glob.o -L. -Lop
enbsd-compat/ -L/home/ssh/ssh_install/openssl-0.9.8h/lib -L/home/ssh/ssh_install/zlib-1.2.3/lib  -lssh -lopenbsd-compat -lcrypto -lutil -
lz -lnsl  -lcrypt -lresolv -lresolv
root@ubuntu:/home/ssh/openssh-4.6p1#
```

编译完成， 最后肯定是需要安装到开发板上的， 如下图所示。 最后在目录下“/home/ssh/openssh-4.6p1” 有这几个二进制文件“scp 、 sftp 、 ssh、 sshd、 ssh-add、 ssh-agent、 ssh-keygen、 ssh-keyscan” 需要安装到开发板上。 最终编译出来的二进制文件， 可以使用 file 命令查看文件属性， 如下图所示， 使用命令“file scp” ， 可以看到“scp” 文件是 32 位 ARM 上使用的可执行文件

```
root@ubuntu:/home/ssh/openssh-4.6p1# file scp
scp: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux.so.3, for GNU/Linux 2.6.16, no
t stripped
root@ubuntu:/home/ssh/openssh-4.6p1#
```

二.安装 ssh

我们把之前我们做好的 QT 的文件系统拷贝到 Ubuntu 上，然后新建一个名为 root 的文件夹，在这个文件夹下解压我们拷贝到 Ubuntu 的 QT 的文件系统或者将没有打包的文件系统放进去。因为我们要把编译好的 ssh 安装到文件系统里面。如下图所示：

```
root@ubuntu:/home/ssh# mkdir root
root@ubuntu:/home/ssh# mv rootfs.tar.bz2 root
root@ubuntu:/home/ssh# cd root/
root@ubuntu:/home/ssh/root# ls
rootfs.tar.bz2
root@ubuntu:/home/ssh/root# tar -xf rootfs.tar.bz2
root@ubuntu:/home/ssh/root# ls
bin  dev  etc  lib  linuxrc  mnt  opt  proc  rootfs.tar.bz2  sbin  sys  tmp  usr  var
root@ubuntu:/home/ssh/root#
```

在解压或者拷贝到的文件系统“/usr/libexec“、“/usr/local/etc”、“/usr/local/bin”三个目录，使用命令“mkdir -p ./usr/libexec ./usr/local/etc ./usr/local/bin”，如下图所示

```
root@ubuntu:/home/ssh/root#
root@ubuntu:/home/ssh/root# mkdir -p ./usr/libexec ./usr/local/etc ./usr/local/bin
```

在虚拟机 Ubuntu 上，将 openssh-6.6p1 下的生成“scp、sftp、ssh、sshd、ssh-add、ssh-agent、ssh-keygen、ssh-keyscan”可执行文件拷贝到文件系统“/usr/local/bin”目录下，拷贝完成，如下图所示

```
root@ubuntu:/home/ssh/root# cp ../openssh-4.6p1/scp ./usr/local/bin/
root@ubuntu:/home/ssh/root# cp ../openssh-4.6p1/sftp ./usr/local/bin/
root@ubuntu:/home/ssh/root# cp ../openssh-4.6p1/ssh ./usr/local/bin/
root@ubuntu:/home/ssh/root# cp ../openssh-4.6p1/sshd ./usr/local/bin/
root@ubuntu:/home/ssh/root# cp ../openssh-4.6p1/sshadd ./usr/local/bin/
cp: cannot stat '../openssh-4.6p1/sshadd': No such file or directory
root@ubuntu:/home/ssh/root# cp ../openssh-4.6p1/ssh-add ./usr/local/bin/
root@ubuntu:/home/ssh/root# cp ../openssh-4.6p1/ssh-agent ./usr/local/bin/
root@ubuntu:/home/ssh/root# cp ../openssh-4.6p1/ssh-keygen ./usr/local/bin/
root@ubuntu:/home/ssh/root# cp ../openssh-4.6p1/ssh-keyscan ./usr/local/bin/
root@ubuntu:/home/ssh/root#
root@ubuntu:/home/ssh/root#
root@ubuntu:/home/ssh/root# ls usr/local/bin/
scp  sftp  ssh  ssh-add  ssh-agent  sshd  ssh-keygen  ssh-keyscan
root@ubuntu:/home/ssh/root#
```

将“moduli、ssh_config、sshd_config”拷贝到开发板的“/usr/local/etc”目录下，拷贝完成，如下图所示。

```
root@ubuntu:/home/ssh/root# cp ../openssh-4.6p1/moduli ./usr/local/etc/
root@ubuntu:/home/ssh/root# cp ../openssh-4.6p1/ssh_config ./usr/local/etc/
root@ubuntu:/home/ssh/root# cp ../openssh-4.6p1/sshd_config ./usr/local/etc/
root@ubuntu:/home/ssh/root# ls usr/local/etc/
moduli  ssh_config  sshd_config
root@ubuntu:/home/ssh/root#
```

将“sftp-server、ssh-keysign”拷贝到开发板的“/usr/libexec”目录下，拷贝完成，如下图所示

```
root@ubuntu:/home/ssh/root# cp ../openssh-4.6p1/sftp-server ./usr/libexec/
root@ubuntu:/home/ssh/root# cp ../openssh-4.6p1/ssh-keysign ./usr/libexec/
root@ubuntu:/home/ssh/root# ls ./usr/libexec/
sftp-server  ssh-keysign
```

（四）生成 KEY 文件，并安装

使用“ssh-keygen” 生成个四个 key 文件“ssh_host_rsa_key”

“ssh_host_dsa_key”

“ssh_host_ecdsa_key” 和“ssh_host_ed25519_key” 。

1 在虚拟机 Ubuntu 控制台， “ /home/ssh/openssh-4.6p1” 目录下， 使用命令“ssh-keygen -t rsa -f ssh_host_rsa_key -N ""” ， 可以生成

“ssh_host_rsa_key ” 文件

如下图所示：

```
root@ubuntu:/home/ssh/openssh-4.6p1# ssh-keygen -t rsa -f ssh_host_rsa_key -N ""
Generating public/private rsa key pair.
Your identification has been saved in ssh_host_rsa_key.
Your public key has been saved in ssh_host_rsa_key.pub.
The key fingerprint is:
SHA256:ZKFPWzv+SupS6IhUeSs28SxQkgBXP1ZOQ7FUSlXOXF4 root@ubuntu
The key's randomart image is:
+---[RSA 2048]----+
|+..o.  .X+o.. . E|
| .o .. B = + o . |
|   o .= B . + .  |
|  . +..* o .     |
|   o = oS o      |
|  . = = .. .     |
| . o * .  o      |
|  . . o  o .     |
|       oo ...    |
+----[SHA256]-----+
```

2 接着使用剩下的三条命令：

ssh-keygen -t dsa -f ssh_host_dsa_key -N ""

ssh-keygen -t ecdsa -f ssh_host_ecdsa_key -N ""

ssh-keygen -t dsa -f ssh_host_ed25519_key -N ""

执行完成之后， 将生成的“ssh_host_rsa_key” “ssh_host_dsa_key”

“ssh_host_ecdsa_key” 和

“ssh_host_ed25519_key” 文件， 拷贝到文件系统的“/usr/local/etc/” 目录下，然后将其权限修改为 600， 如下图所示：

```
root@ubuntu:/home/ssh/openssh-4.6p1# cp ssh_host_rsa_key ../root/usr/local/etc/
root@ubuntu:/home/ssh/openssh-4.6p1# cp ssh_host_dsa_key ../root/usr/local/etc/
root@ubuntu:/home/ssh/openssh-4.6p1# cp ssh_host_ecdsa_key ../root/usr/local/etc/
root@ubuntu:/home/ssh/openssh-4.6p1# cp ssh_host_ed25519_key ../root/usr/local/etc/
root@ubuntu:/home/ssh/openssh-4.6p1# ls ../root/usr/local/etc/
moduli  ssh_config  sshd_config  ssh_host_dsa_key  ssh_host_ecdsa_key  ssh_host_ed25519_key  ssh_host_rsa_key
root@ubuntu:/home/ssh/openssh-4.6p1#
```

然后我们回到文件系统的解压目录，使用 tar 命令将文件系统打包，如果是其他格式的文件系统，如 img，按照具体开发板使用手册的工具打包即可。如下图所示：

```
root@ubuntu:/home/ssh/root# ls
bin  dev  etc  lib  linuxrc  mnt  opt  proc  rootfs.tar.bz2  sbin  sys  tmp  usr  var
root@ubuntu:/home/ssh/root# rm rootfs.tar.bz2
root@ubuntu:/home/ssh/root#
root@ubuntu:/home/ssh/root#
root@ubuntu:/home/ssh/root#
root@ubuntu:/home/ssh/root#
root@ubuntu:/home/ssh/root# ls
bin  dev  etc  lib  linuxrc  mnt  opt  proc  sbin  sys  tmp  usr  var
root@ubuntu:/home/ssh/root#
root@ubuntu:/home/ssh/root# tar -cjf rootfs.tar.bz2 ./*
root@ubuntu:/home/ssh/root# ls
bin  dev  etc  lib  linuxrc  mnt  opt  proc  rootfs.tar.bz2  sbin  sys  tmp  usr  var
```

然后我们将打包完的文件系统烧写到开发板。

二．修改目标板秘钥文件

在文件系统中， 使用命令“vi /etc/passwd” ， 打开 ssh 密钥文件， 在“/etc/passwd” 文件底行添

加以下内容：

sshd:x:74:74:Privilege-separated

SSH:/var/empty/sshd:/sbin/nologin

添加完成之后， 如下图所示， 然后保存， 密钥文件修改完成

```
root::0:0:root:/:/bin/sh
bin:*:1:1:bin:/bin:
daemon:*:2:2:daemon:/sbin:
nobody:*:99:99:Nobody:/:
sshd:x:74:74:Privilege-separated
SSH:/var/empty/sshd:/sbin/nologin
```

三．测试

保证开发板和虚拟机是可以 ping 通的前提下，在开发板上新建 root 账户， 如下图所示， 使用命令“passwd root” ， 它会提示输入密码， 需要重复输入两次以防输错

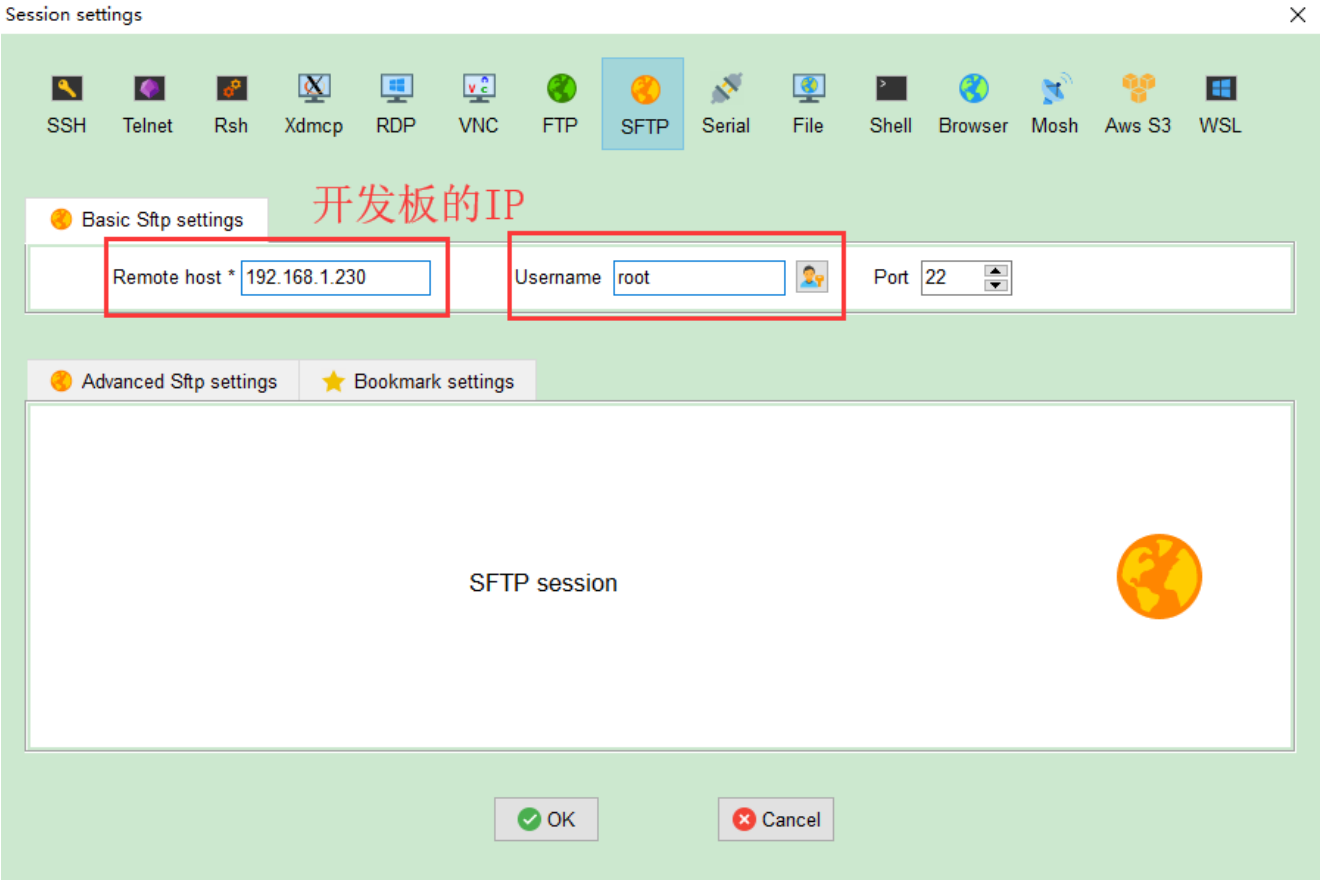
```
[root@iTOP-iMX6UL]# passwd root
Changing password for root
New password:
Bad password: too weak
Retype password:
passwd: password for root changed by root
```

然后在开发板上运行 sshd 二进制文件， 使用命令“/usr/local/bin/sshd” ， 提示如下

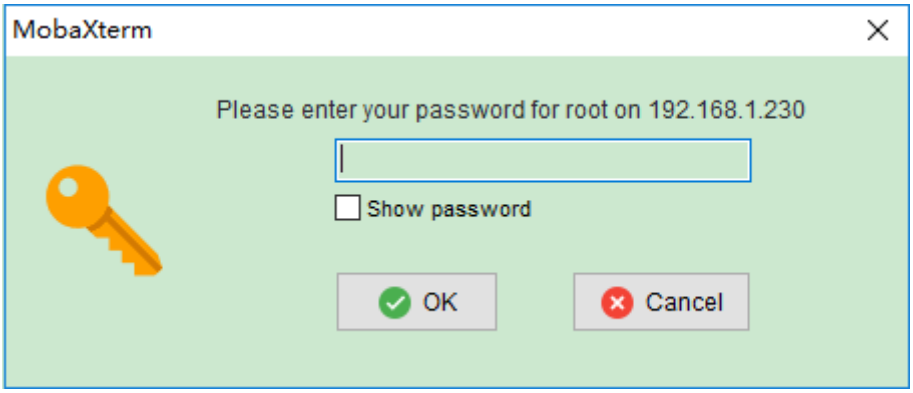
```
[root@iTOP-iMX6UL]# /usr/local/bin/sshd
Could not load host key: /usr/local/etc/ssh_host_key
Disabling protocol version 1. Could not load host key
```

虽然提示出错了，但是作者实际测试并不影响使用， 所以我们先跳过这个错误。

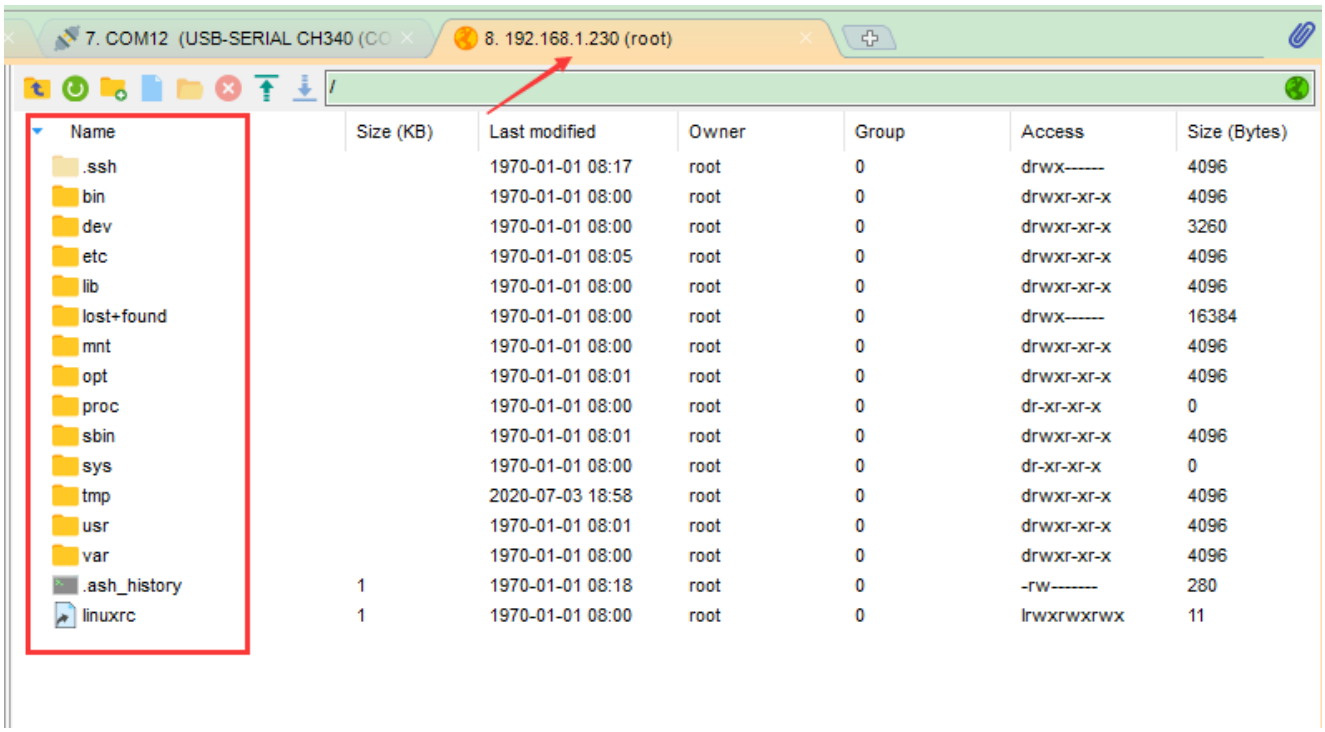
然后使用工具 ssh 测试是否和 PC 联通能否正常通信，我们打开 ssh 工具，然后输入开发板的 IP 和用户名，如下图所示：



根据软件的提示输入我们之前设置的密码：



可以发现我们已经成功的连接上了开发板。如下图所示，这样我们的 ssh 就移植成功了。



二．在 Ubuntu 上搭建 QT 开发环境。

为了避免由于开发环境的版本差异导致的不必要的问题， 推荐用户下载和使用手册中安装的相同版本“qtcreator-3.5.1” 。 用户有两种方式可以下载。 这里需要注意的是，如果用户想要在上位机上调试， 则需要下载对应环境的插件。 我们提供的是“qt-opensource-linux-x64-5.5.1.run” ， 里面包含了在 Ubuntu 中运行的插件， 也就是编译出来的应用程序， 可以在 Ubuntu16.04.6 中运行。 然后经过简单的移植， 就可以在开发板上运行。

官网下载地址 http://download.qt.io/archive/qt/5.5/5.5.1/，

（1）安装 qtcreator-3.5.1

我们将 qt-opensource-linux-x64-5.5.1.run 安装包通过 ssh 传到 Ubuntu 的/opt 目录里面，如下图所示：

```
root@ubuntu:/opt#
root@ubuntu:/opt# ls
fsl-imx-x11   qt5.7.0                          qt-opensource-linux-x64-5.5.1.run  tslib1.4
opencv-3.4.1  qt-everywhere-opensource-src-5.7.0  tslib                              vmware-tools-installer
root@ubuntu:/opt#
```

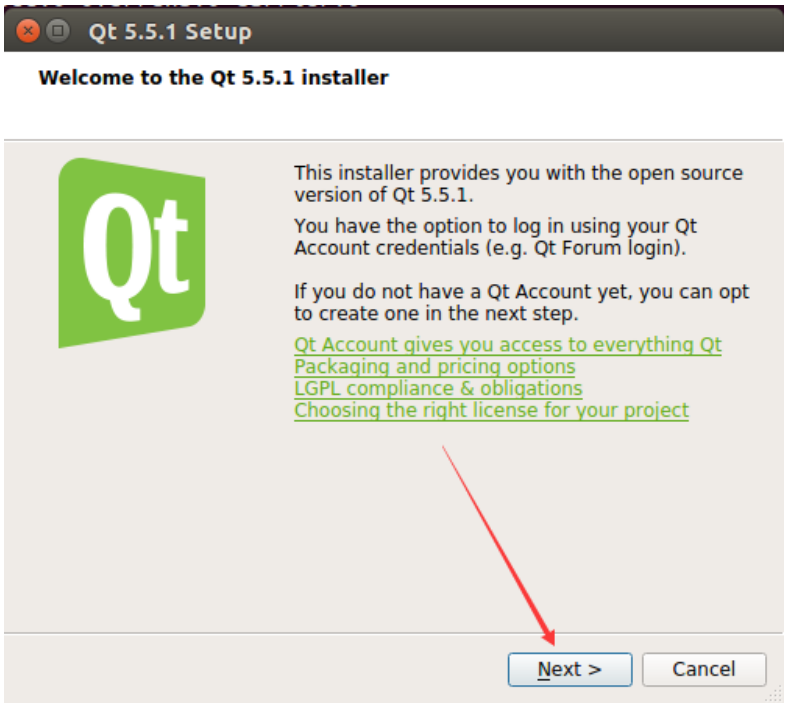
(2) 然后我们在 ubuntu 下打开终端， 切换为 root 用户， 我们使用命令“chmod 777 qt-opensource-linux-x64-5.5.1.run” 修改安装包权限。 如下图所示

```
root@ubuntu:/opt# chmod 777 qt-opensource-linux-x64-5.5.1.run
root@ubuntu:/opt# ls
fsl-imx-x11   qt5.7.0                          qt-opensource-linux-x64-5.5.1.run  tslib1.4
opencv-3.4.1  qt-everywhere-opensource-src-5.7.0  tslib                              vmware-tools-installer
root@ubuntu:/opt#
```

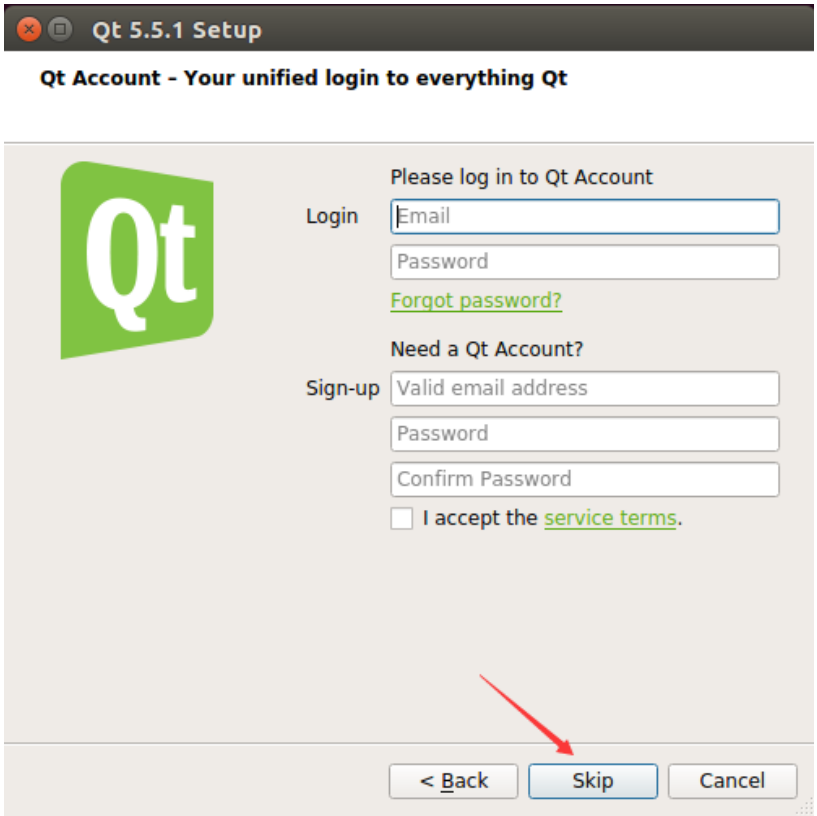
(3) 接下来使用命令“./qt-opensource-linux-x64-5.5.1.run” 执行安装。 如下图所示

```
root@ubuntu:/opt# ./qt-opensource-linux-x64-5.5.1.run
qt.network.ssl: QSslSocket: cannot resolve SSLv2_client_method
qt.network.ssl: QSslSocket: cannot resolve SSLv2_server_method
```

(4) 欢迎界面点击“next” 继续。 如下图:

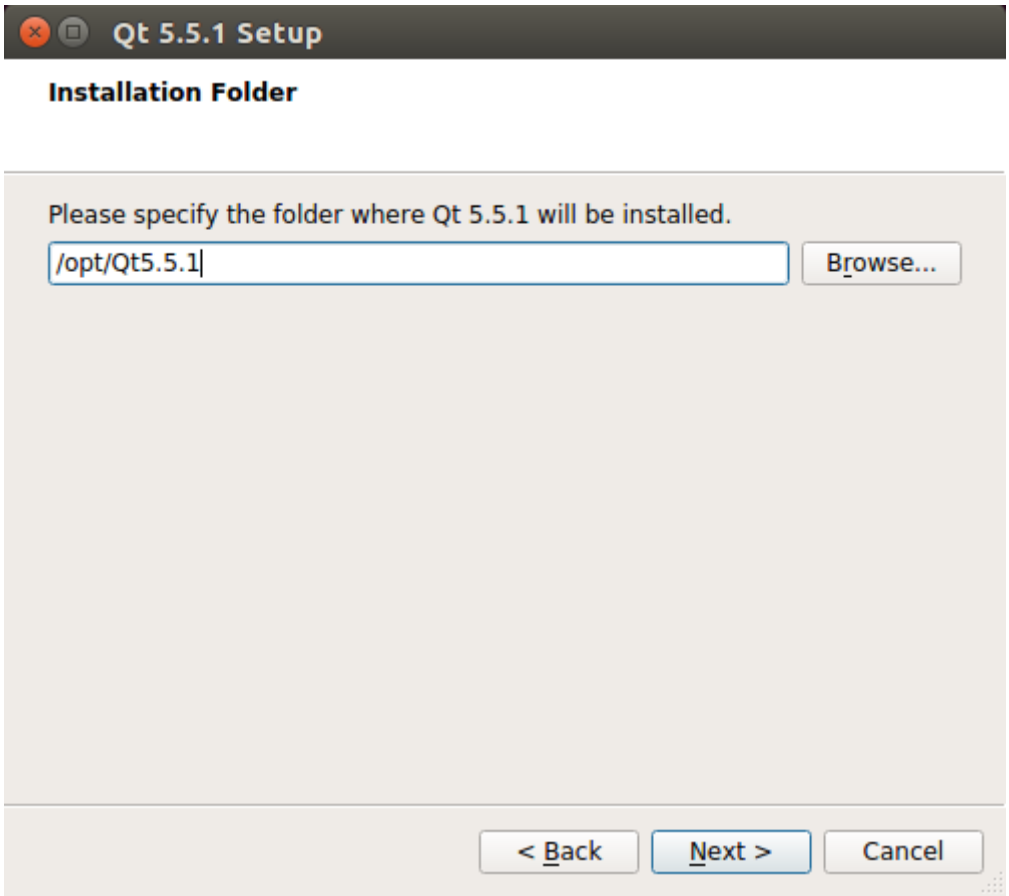


(5)然后到输入账号密码界面， 点击“ skip” 跳过。 如下图

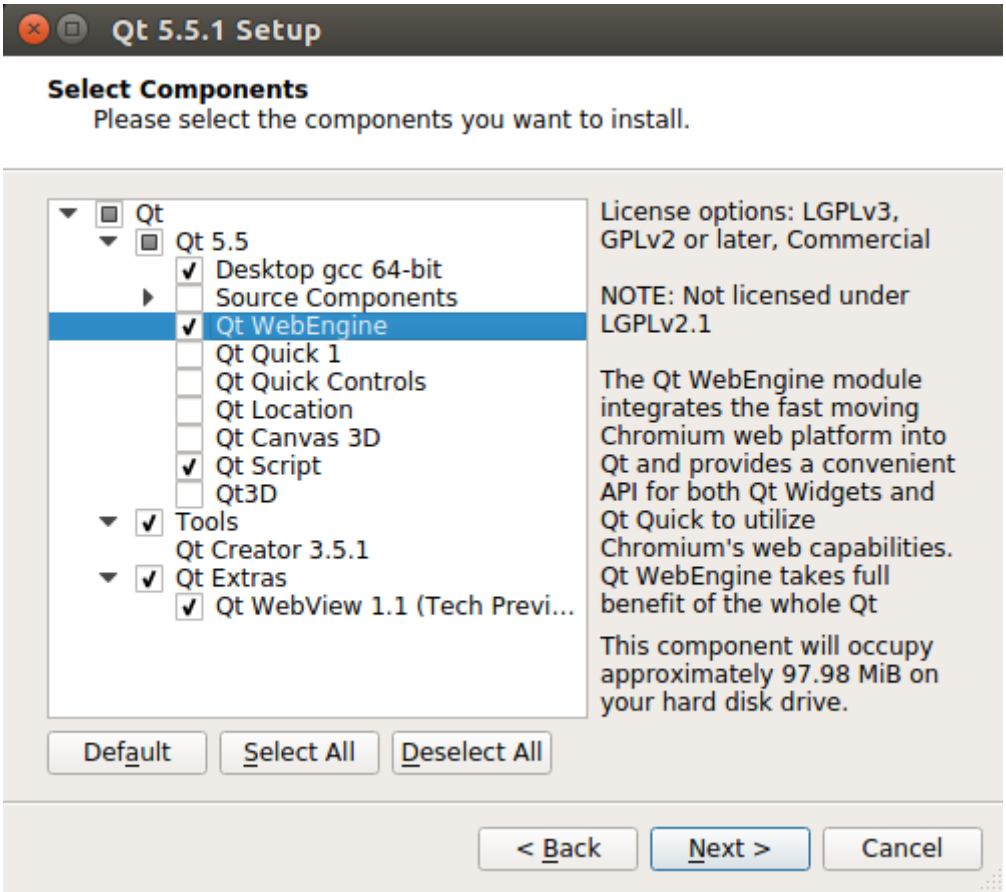


(5) 继续点击“ next” 下一步。 如下图：

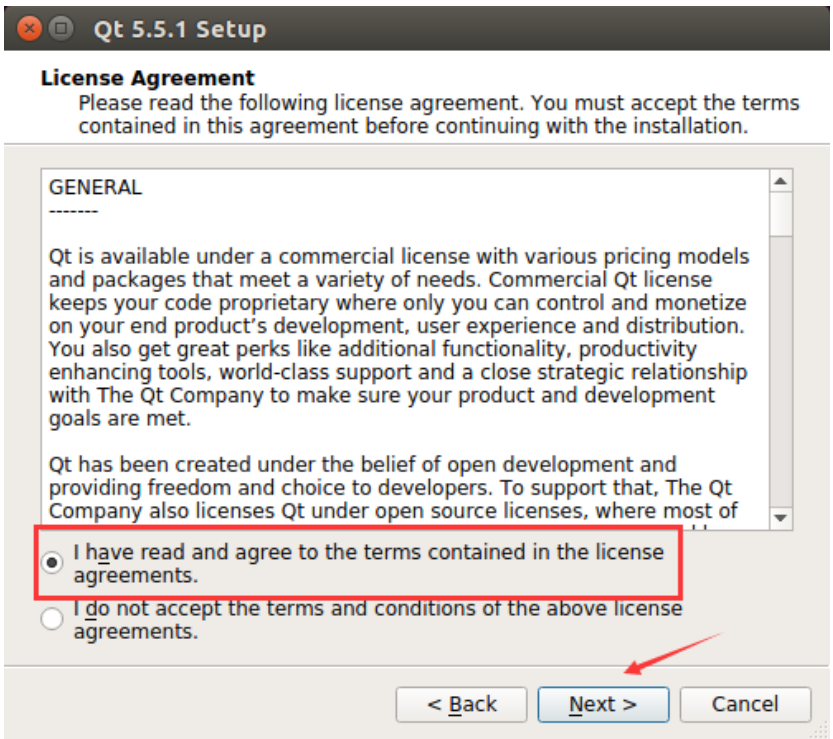
(6)然后选择安装路径， 我们不用修改（ 后面我们好多地方会用到这个路径， 尽量不要修改） ， 点击“next” 下一步。



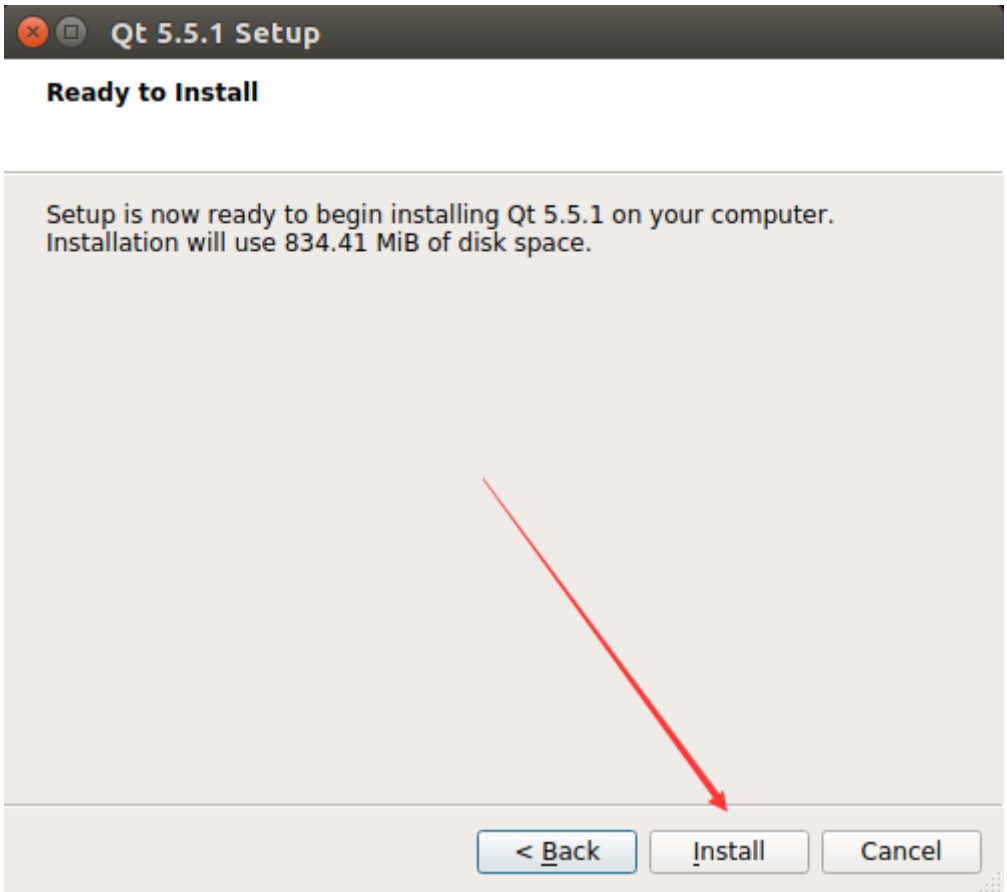
(7)接下来我们选择安装的组件， 勾选图中几个选项即可， 如下图所示：



(8)然后选择第一项已同意并阅读条款。 再点击下一步。 如下图

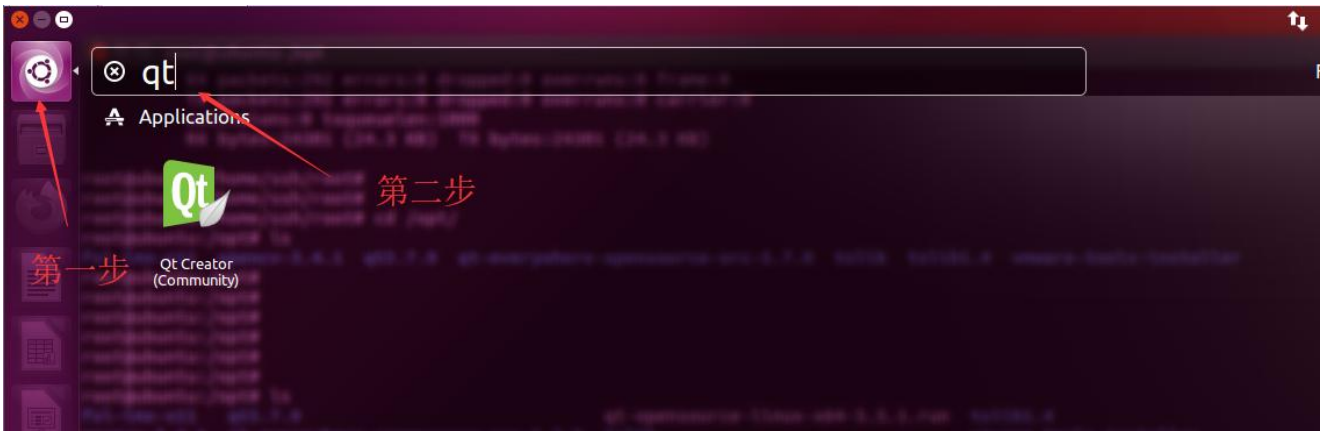


(9)然后点击“install” 安装 qt， 如下图



(10)等待装完成， 点击“ finish” 结束安装即可。 如下图

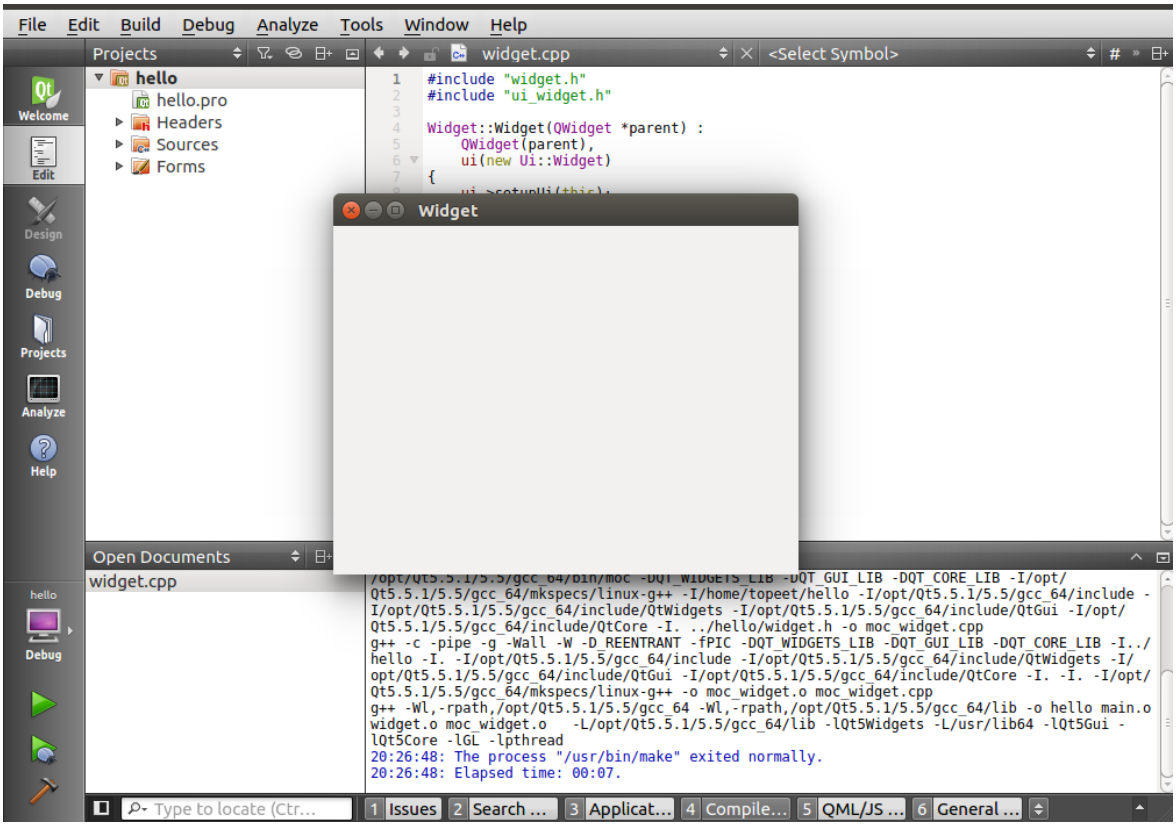
(11) 安装完成后我们可以通过“ search your computer” 搜索 qt。 如



(12) 打开 QT 之前，我们需要安装一些必要的库文件。

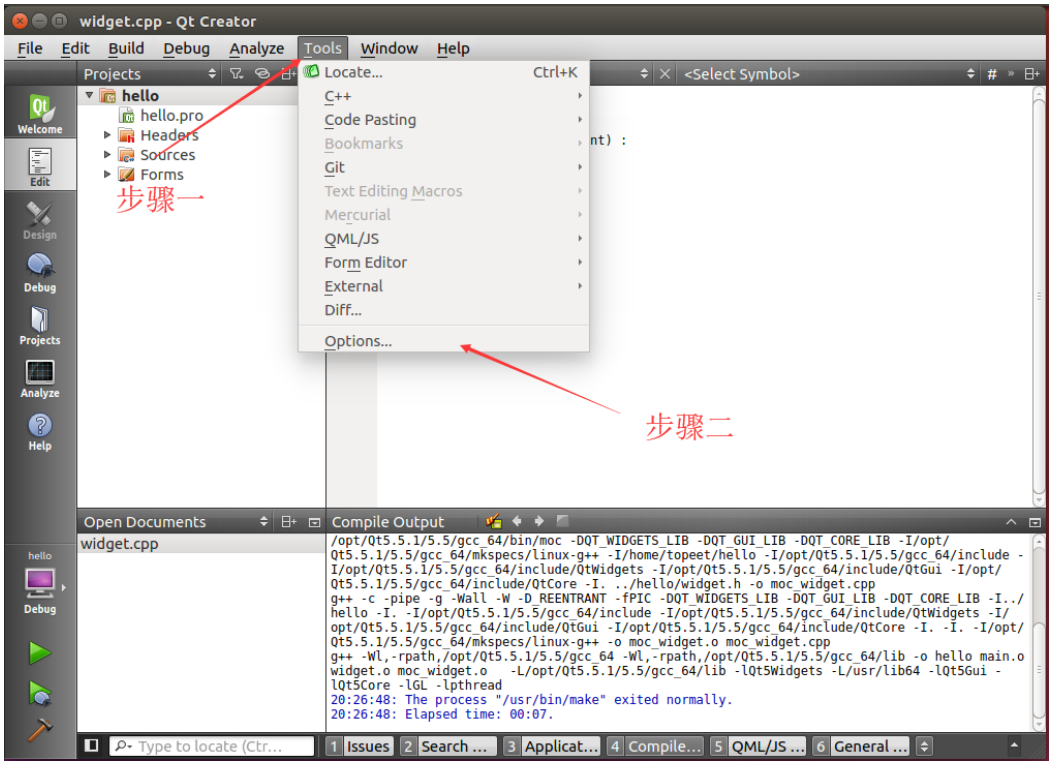
```
apt-get install libqt4-dev
apt-get install libgstreamer0.10-dev
apt-get install libgstreamer-plugins-base0.10-dev
```

(13) 安装好依赖以后，我们打开 QTcreator，然后新建一个工程，创建工程的方法和在 Windows 上使用 QTcreator 创建的方法一样，这里就不在赘述了。创建一个以后，我们点击一下编译，如果可以编译成功，则说明我们的环境搭建正确，如下图所示：

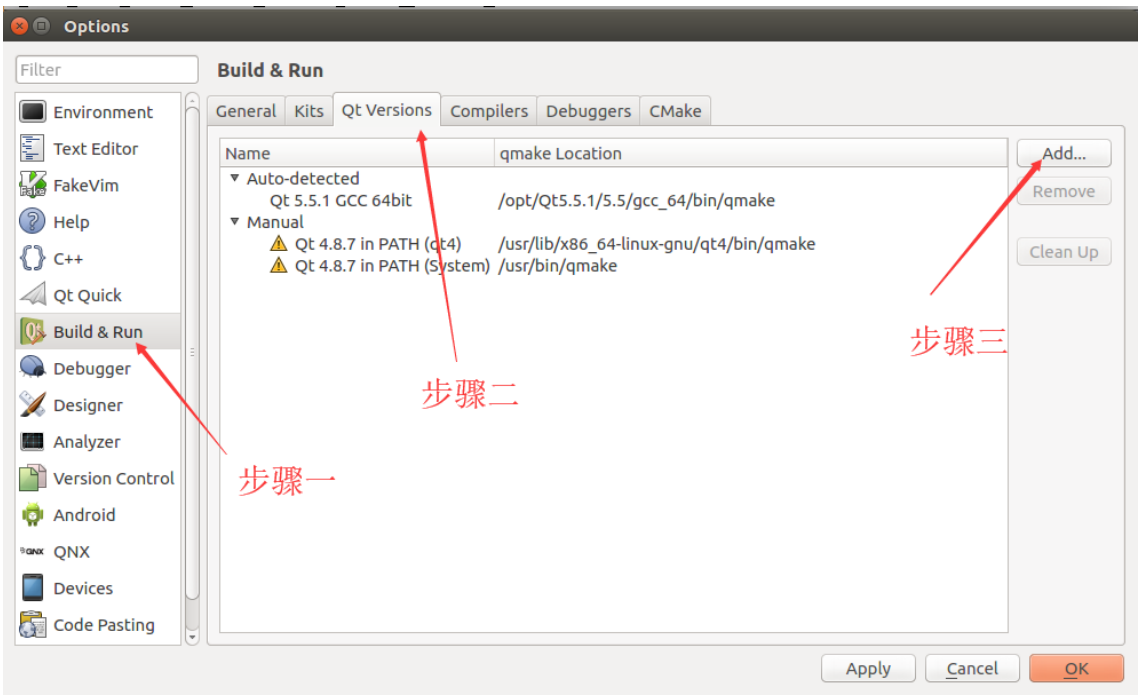


三．配置编译套件

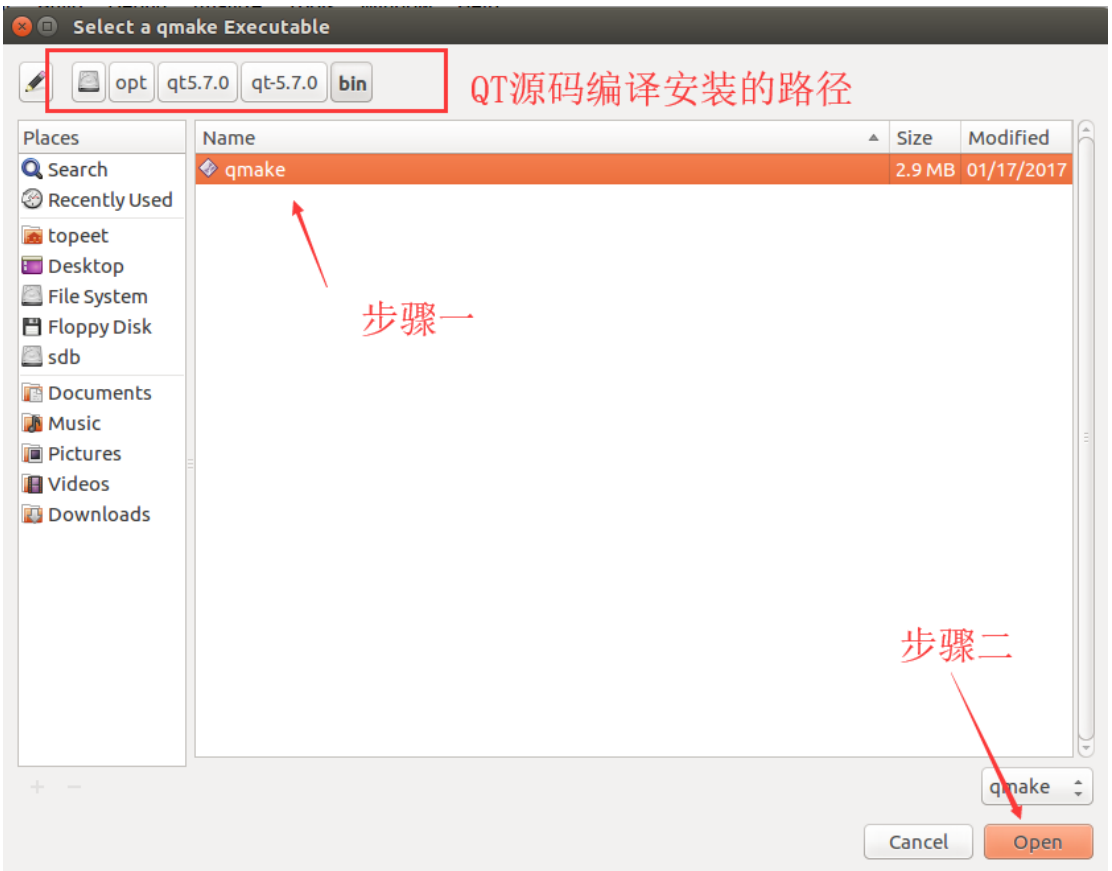
（1）点击菜单栏中的 Tools->选择 Options，如下图所示：



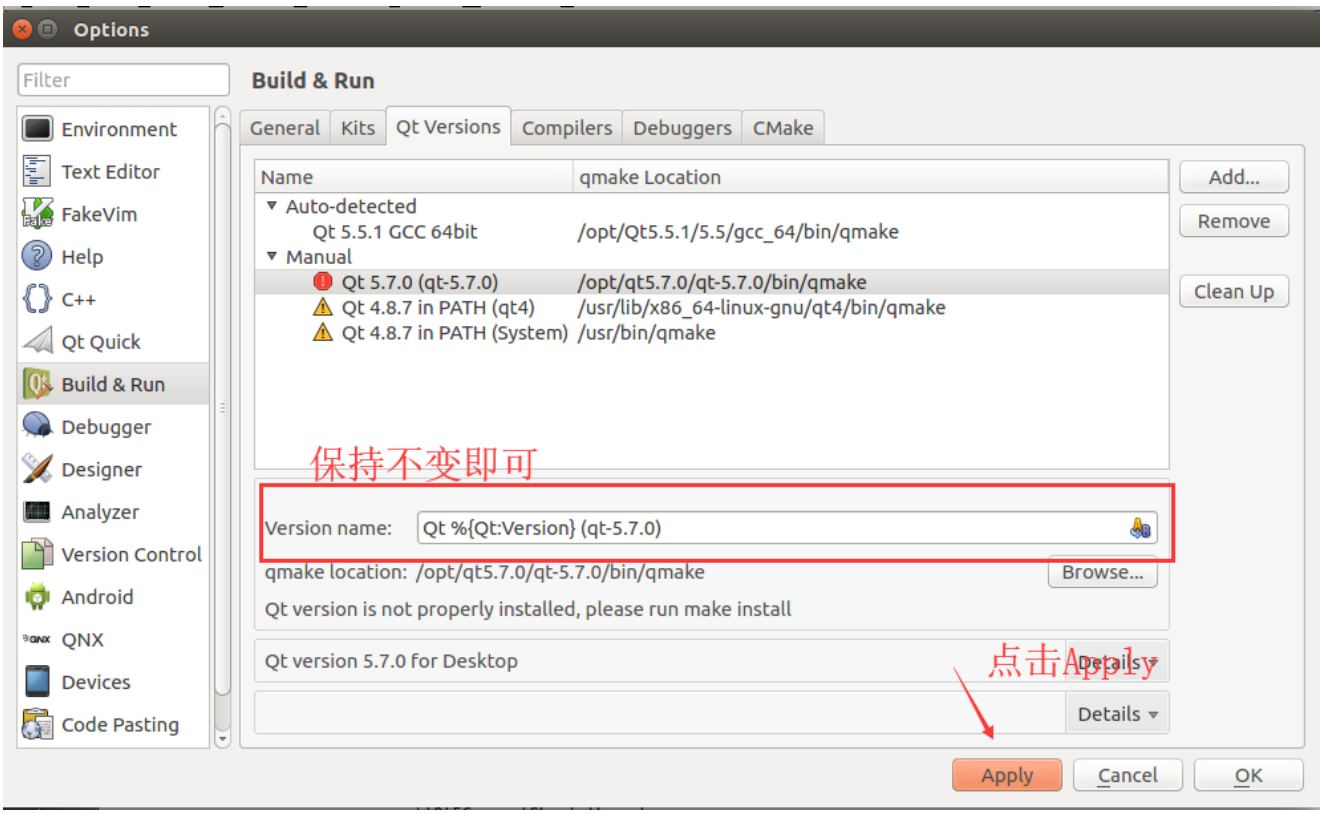
（2）然后点击 Build&Run，选择 QT Versions，点击 add，如下图所示



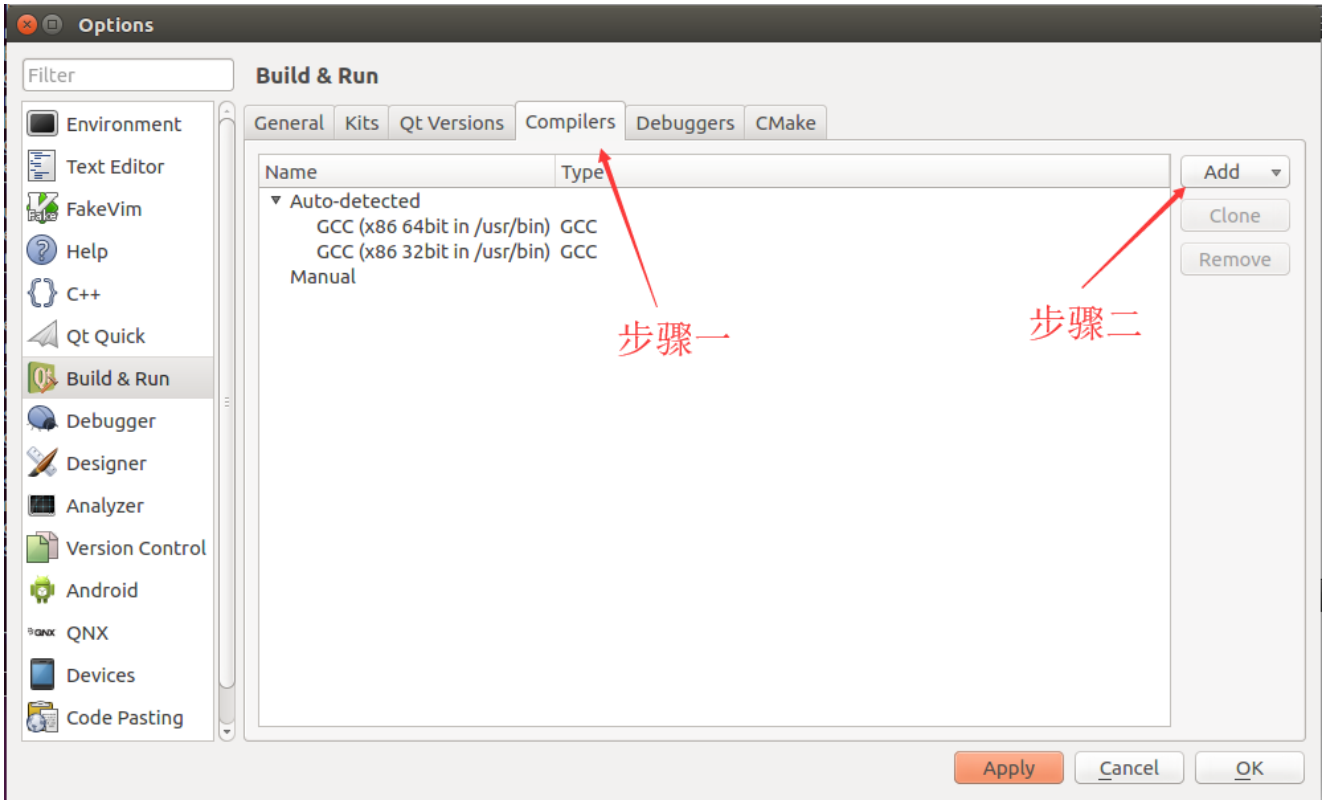
（3）在弹出来的窗口，我们选择编译 QT 源码生成的 qmake，选中以后点击 open，如果大家不清楚要怎么编译 QT 源码，请看交叉编译 QT 源码章节，如下图所示：



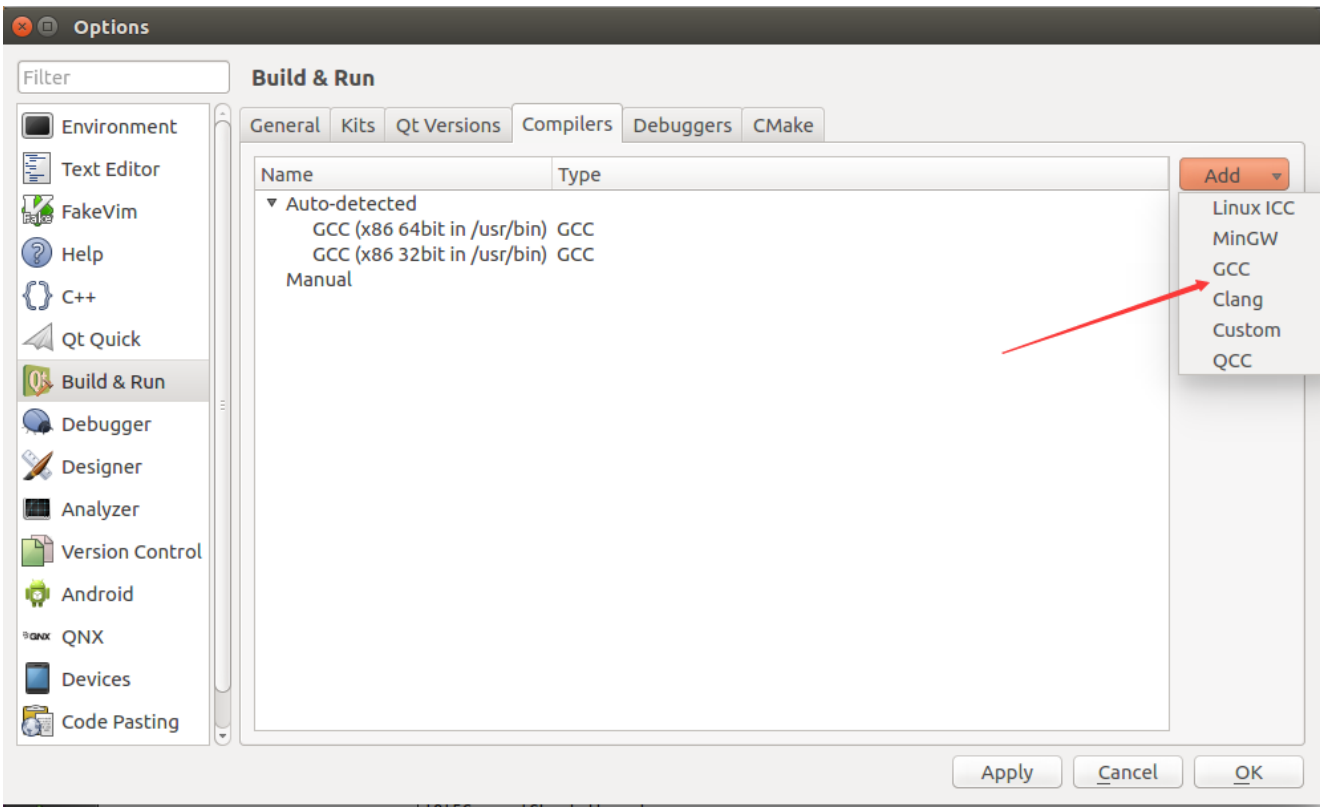
（4）然后点击 Apply，QT 版本就设置完成了。



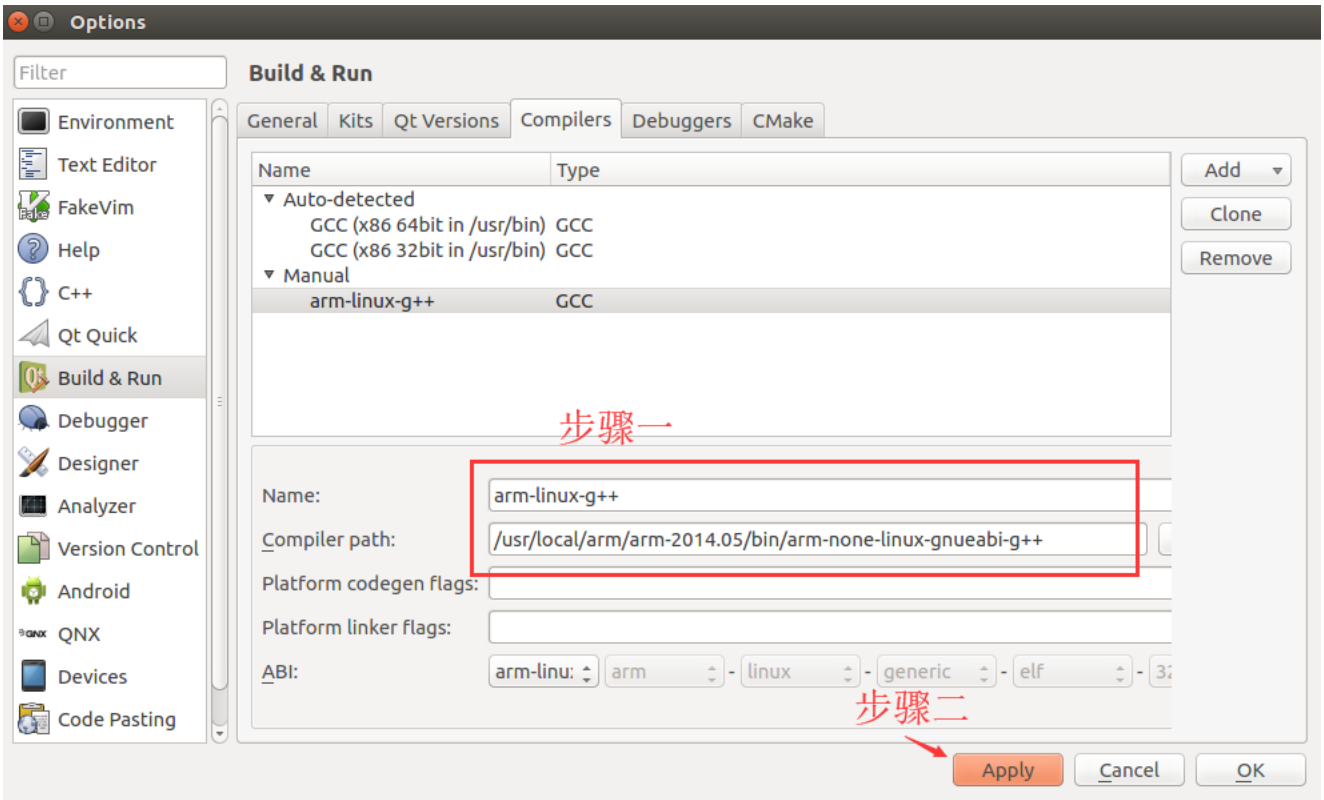
（5）接着我们点击 Compilers，点击里面的 Add，如下图所示：



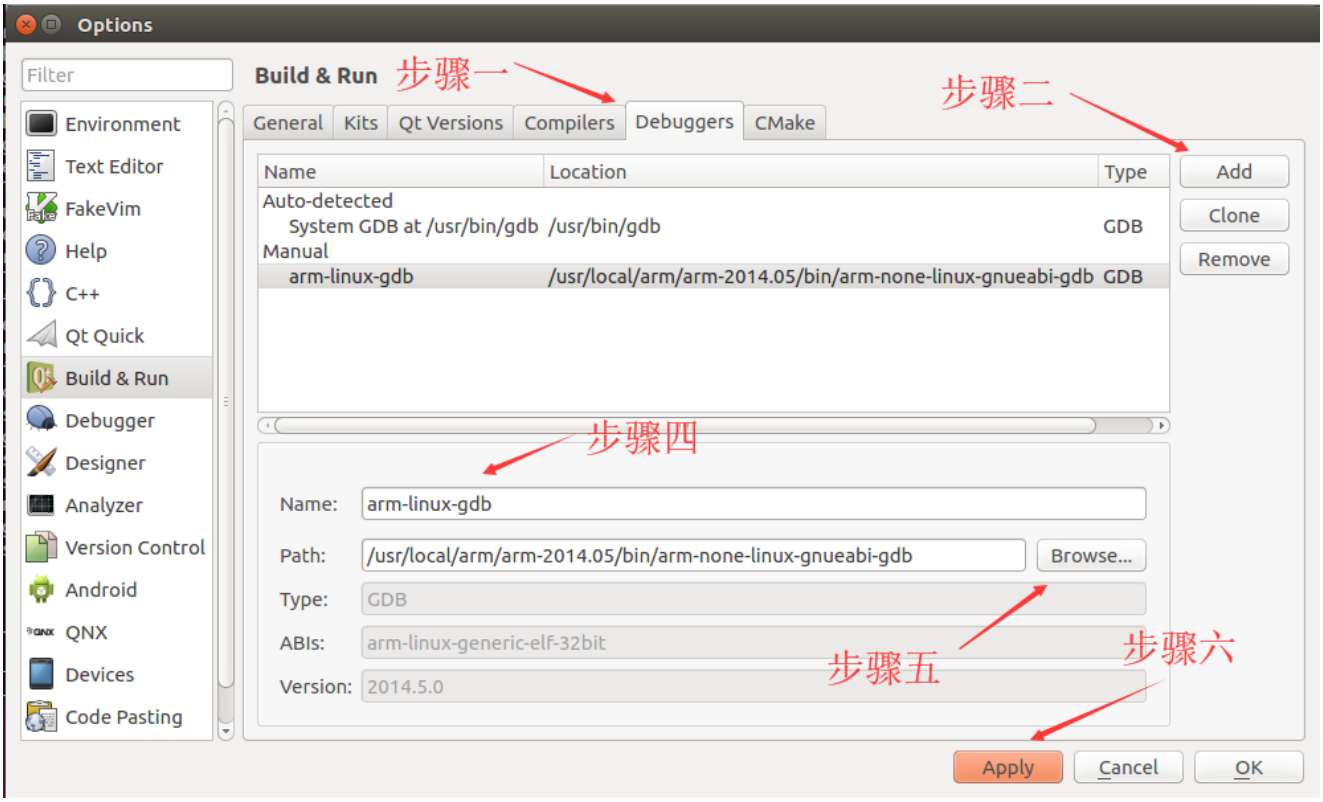
（6）接着我们选择里面的 GCC，如下所示：



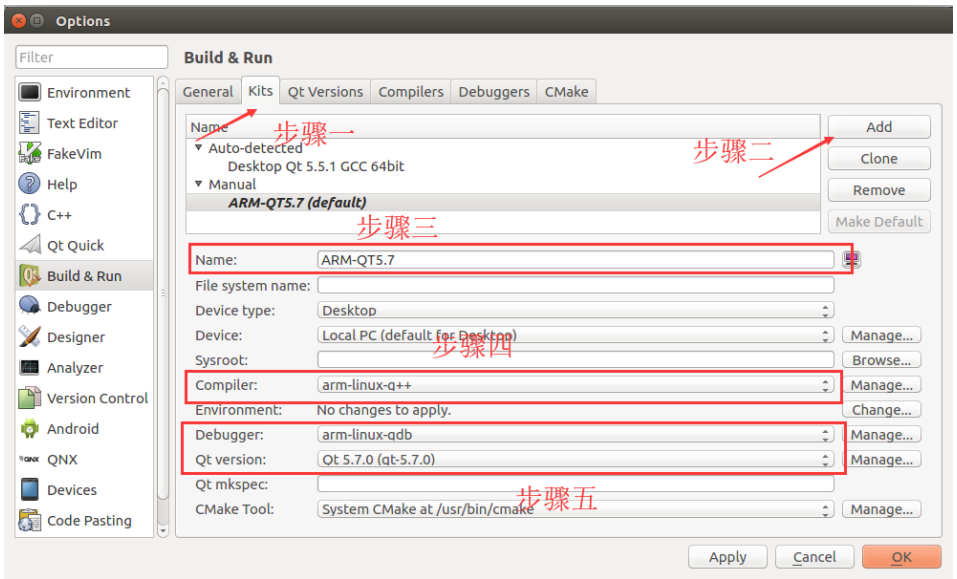
（7）接着我们在 Name 那些写上 arm-linux-gcc，编译器目录选择编译 QT5.7 源码的编译器里面的 g++，然后点击 Apply，如下图所示：



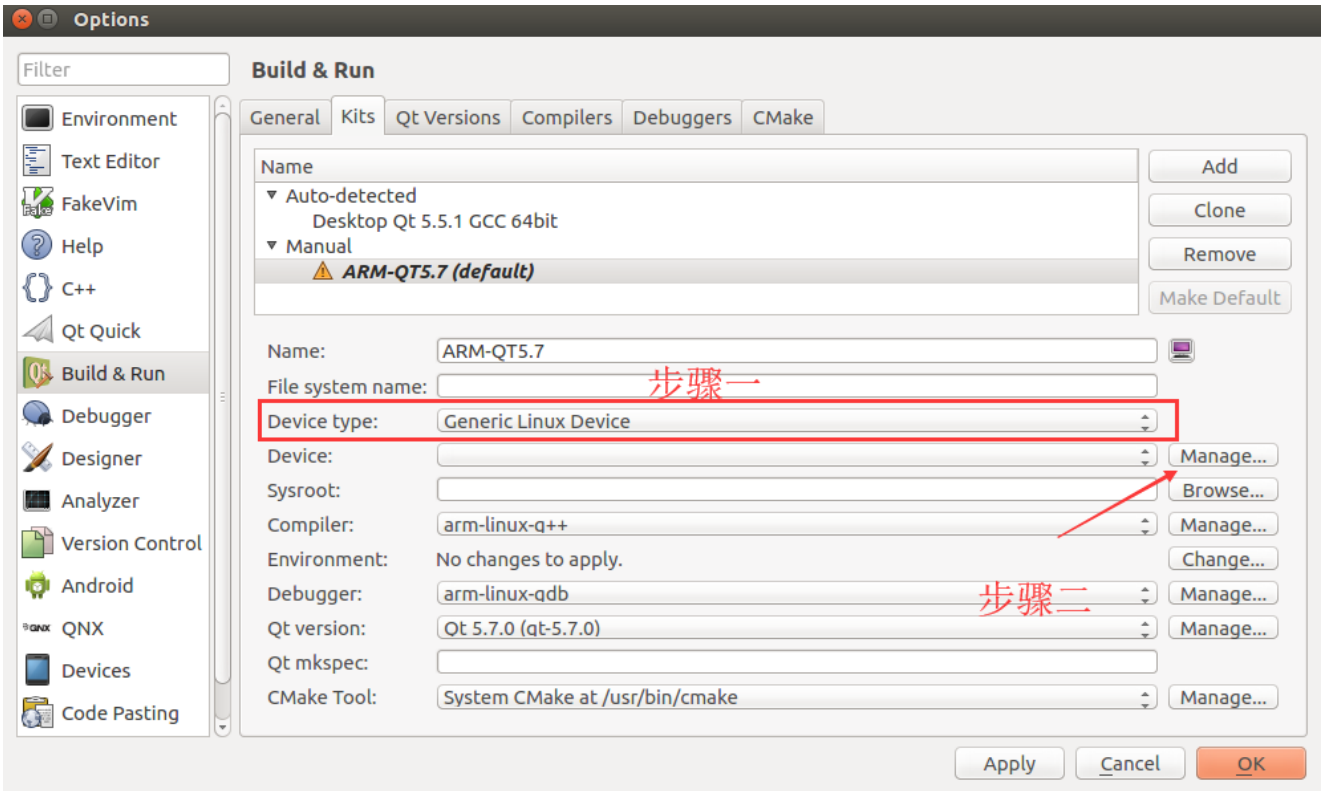
（8）接着来设置 gdb，我们点击 Debuggers，选择 add，然后把 Name 写上 arm-linu-gdb，PATH 的路径选择我们编译 QT 源码的编译器所在的路径，并选择这个编译器的 gdb，如下图所示：然后点击 Apply。



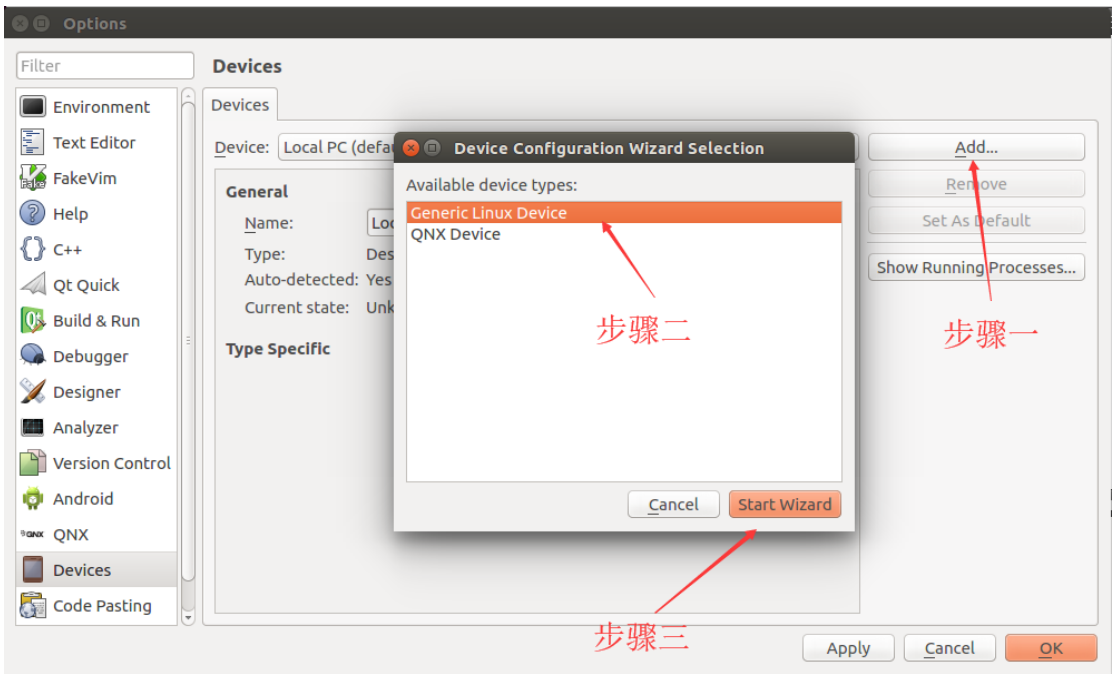
（9）然后我们点击 Kits，点击里面的 add，在弹出的设置界面，我们把 Name 改成 ARM-QT5.7，Compiler 选项里面选择我们之前设置 arm-linux-g++，Debugger 里面选择我们之前设置的 arm-linux-gdb，QT version 里面选择我们新添加的 QT 版本 QT5.7，如下图所示：



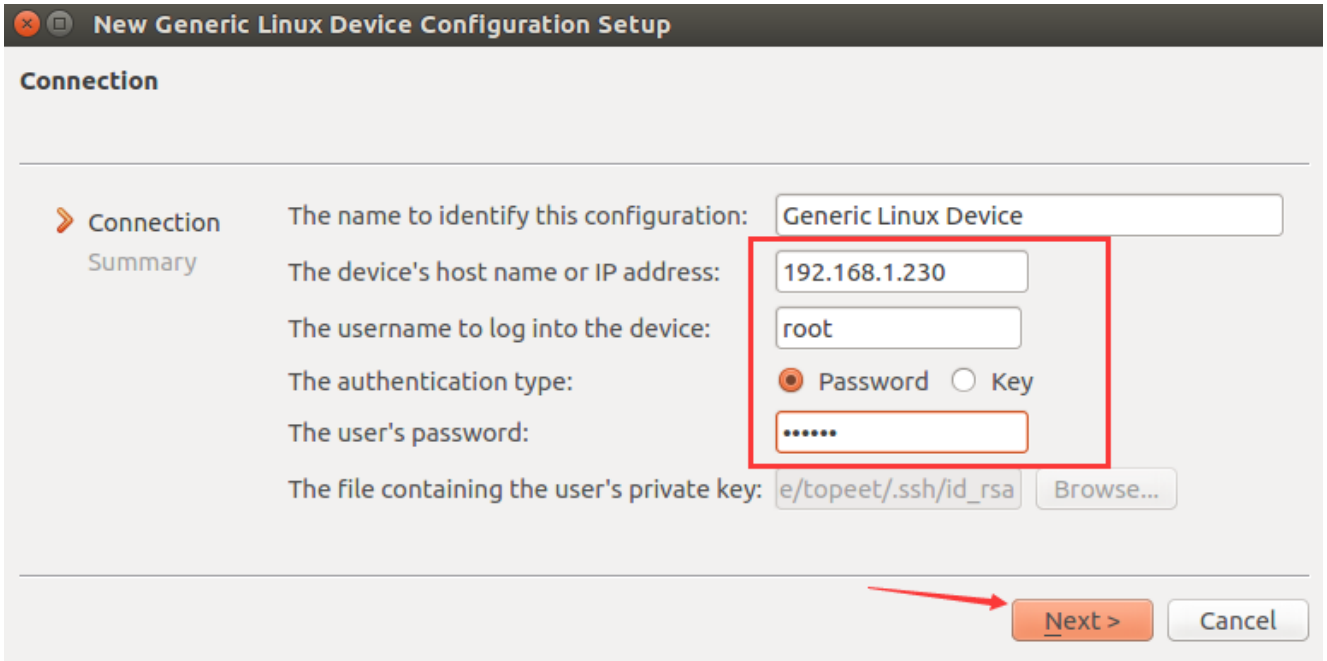
（10）接着我们把 Device type 设置成 Generic Linux Device，然后点击 Device 里面的 Manage，如下图所示：



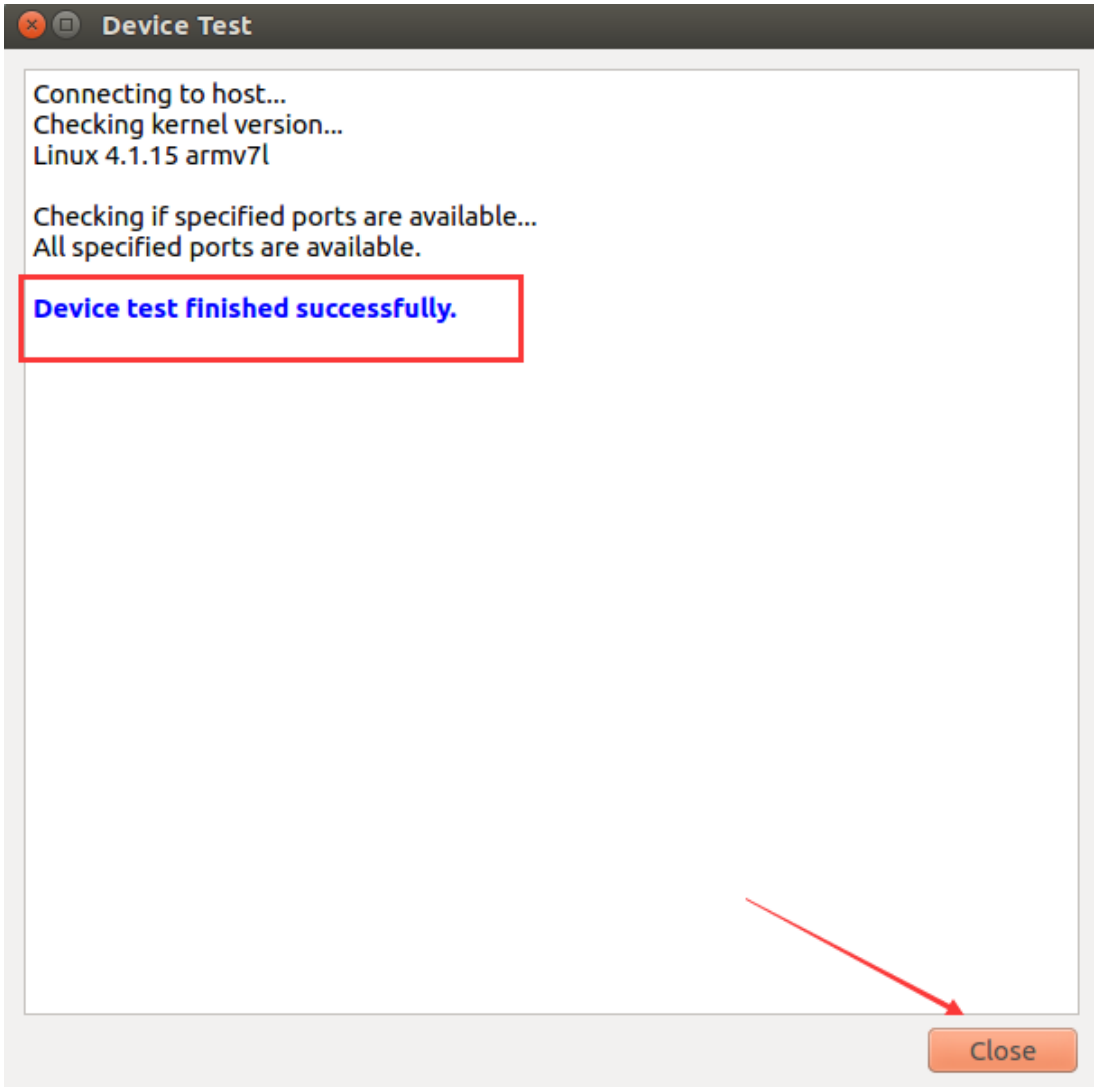
（11）然后在弹出来的界面我们点击 add，选择 Generic Linux Device，选择完成以后点击 Start Wizard，如下图所示：



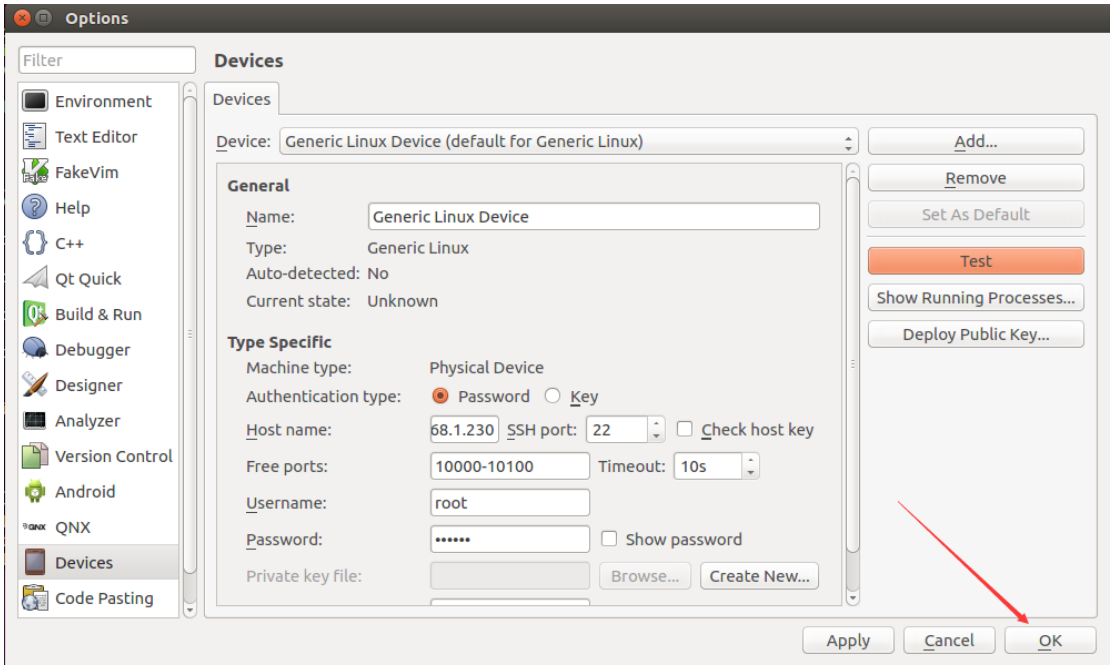
（12）在弹出来的界面我们把 IP 设置成开发板的 IP，username 设置成 root，密码设置成我们在移植 ssh 里面设置的密码，然后点击 next，如下图所示：



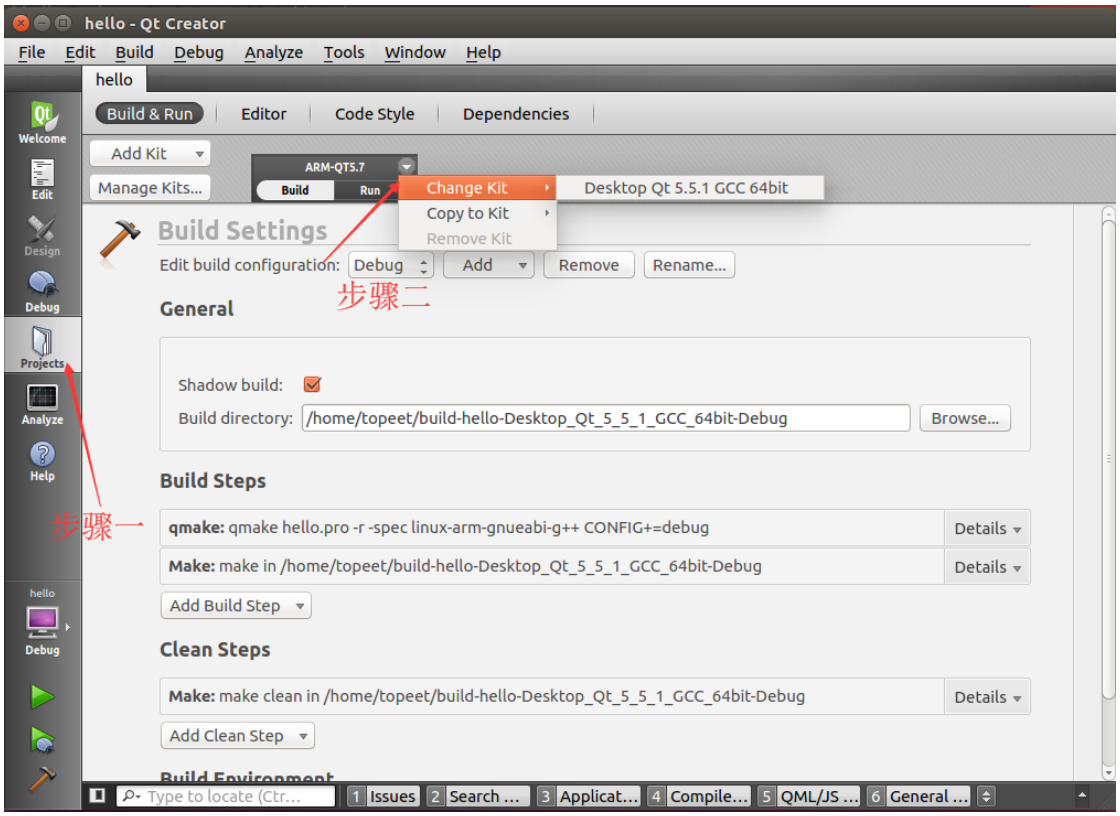
（13）在弹出来的界面，我们点击 finish，如图出现成功的字样，则说明已经设置成功，如果不成功，请检查使用 ssh 软件是不是可以跟开发板连接，还有上一步的设置是不是正确的。如下图所示：



（14）点击 ok，退出编译器设置界面，如下图所示：



（15）然后我们点击 qtcreator 里面的 Projects，把编译器切换成我们的刚才设置的编译器，如下图所示：

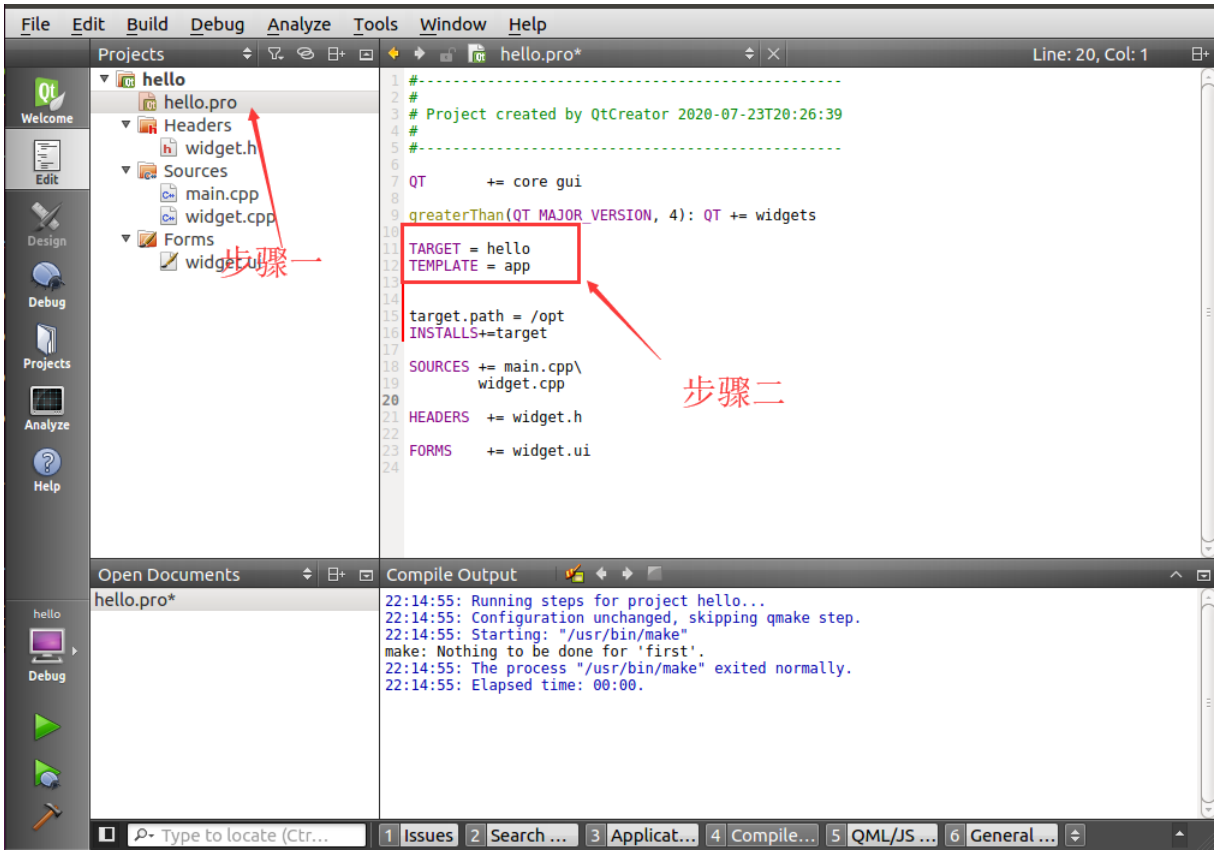


（16）然后我们打开在我们要编译的 QT 代码的 pro 文件里面添加以下代码，

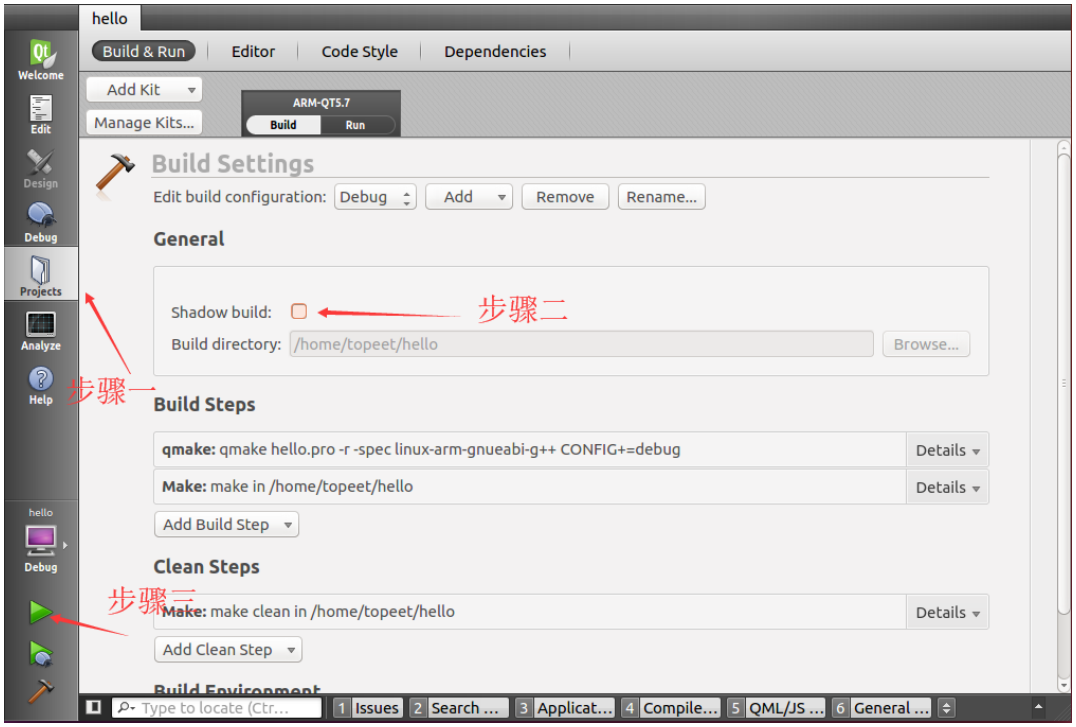
这俩行代码的意思是说把我们的可执行程序下载到开发板的/opt 目录下并执行。

```
target.path=/opt
INSTALLS+=target
```

如下图所示：



（17）然后取消掉构建的路径，接在我们点击一下编译，可以在我们的开发板的屏幕上看到我们 QT 程序直接在开发板运行了，


hello
Build & Run
Editor
Code Style
Dependencies
Welcome
Edit
Design
Debug
Projects
Analyze
Help
Add Kit
Manage Kits...
ARM-QT5.7
Build
Run
Build Settings
Edit build configuration: Debug
Add
Remove
Rename...
General
Shadow build:
步骤二
Build directory: /home/topeet/hello
Browse...
步骤一
Build Steps
qmake: qmake hello.pro -r -spec linux-arm-gnueabi-g++ CONFIG+=debug
Details
Make: make in /home/topeet/hello
Details
Add Build Step
Clean Steps
步骤三
Make: make clean in /home/topeet/hello
Details
Add Clean Step
Type to locate (Ctr...
1 Issues
2 Search ...
3 Applicat...
4 Compile...
5 QML/JS ...
6 General ...